Algorithmique Avancée et Programmation

Serigne A. Gueye
CERI, Licence 2, informatique.

13 novembre 2025

1 Introduction
- Algorithmique Avancée et Programmation
- Plan du cours

2 Chapitre 1 : Arbres
- Définition d'un arbre
- Représentation et codage C++
- Types de parcours
- Propriétés
- Arbres Binaires de Recherche (ABR)
- Opérations fondamentales dans un ABR
- Arbres Binaires de Recherche Equilibrés AVL

3 Chapitre 2. Tas (Heap) : heap sort

4 Chapitre 3. Tris

5 Chapitre 4 : Complexité algorithmique

6 Chapitre 5. Backtracking

Algorithmique Avancée et Programmation

Définition

Un algorithme se définit comme une suite finie d'opérations élémentaires pour la résolution d'un problème donné.

Définition

L'algorithmique est l'ensemble des méthodes et des techniques permettant de **concevoir** et **d'analyser** un algorithme.

Algorithmique Avancée et Programmation

Définition

L'analyse d'un algorithme consiste :

- à prouver que la méthode proposée résoud bien le problème donné en entrée (**preuve de l'algorithme**),
- à évaluer le temps d'éxecution et l'espace mémoire requis (**complexité en temps et en espace**).

Définition

Un programme informatique est le codage d'un algorithme dans un langage informatique donné (C, C++, Java, Python,...).

Ecriture des algorithmes

- Un algorithme doit pouvoir être programmé **dans n'importe quel langage informatique**.
- Nous décrirons les algorithmes par des "pseudo-codes".
- Les implémentations se feront en C++ en TP.

Plan du cours

	Chapitre	CM	TD	TP
1	Arbres	2	2	2
2	Tas	1	1	1
3	Tris	1	1	1
	CC1			
3	Complexité algorithmique	1	1	1
4	Backtracking, min-max	1	1	1
5	Programmation dynamique	1	2	2
6	Tables de Hachage	1	1	1
	CC2			

TP : CODES C++ : LOCALEMENT OU SUR CASEINE

NOTE AMS ALGORITHMIQUE AVANCEE ET GRAPHES :

0.1 * TP (S3) + 0.2 * CASEINE 1 (1h) + 0.3 * CASEINE 2 (2h15) + 0.4 * CASEINE 3 (2h15).

NOTE AA : 0.50*CC1 (ECRIT) + 0.50*CC2 (ECRIT)

Définitions

Définition

Un arbre est un ensemble de noeuds organisés de façon hiérarchique à partir d'un noeud appelé *racine*.

Définitions

Définition

Un arbre, enraciné en r, à **ramification bornée** peut être défini récursivement comme suit

$$A = \emptyset \text{ ou } A =< r, F_1, F_2, ..., F_k >$$

où 'r' est un noeud appelé racine et F_1, F_2,...,F_k sont des arbres disjoints appelés **sous-arbres** de A.

Les noeuds contiennent plusieurs champs dont une étiquette (identifiant) nommée "cle" ($r.cle$).

Arbre binaire, $k = 2$

- F_1 : **sous-arbre gauche**, sa racine est **le fils gauche** de r.
- F_2 : **sous-arbre droit**, sa racine est **le fils droit** de r.
- **Noeud simple** : noeud interne n'ayant qu'un fils.
- **Noeud double** : noeud interne ayant deux fils.

Représentation des arbres binaires

noeud	
cle	un identifiant
pere	adresse du noeud père
fg	adresse du noeud dit fils gauche
fd	adresse du noeud dit fils droit

Pour un noeud x, x.cle, x.pere, x.fg, x.fd désignent les champs.

Codage en c++

- ```
  class noeud
  {
  friend class AB ;
          int cle ;
          noeud * fg ;
          noeud * fd ;
          noeud * pere ;
  public :
          noeud(int x) ;
          ~noeud() ;
  ...
  } ;
  ```

"friend class AB" : toutes les méthodes de la classe AB peuvent accèder aux attributs privés de "noeud".
  ```

Codage en c++

- class AB
  {
  noeud * r ;
  public :
  AB(noeud* r) ;
  ∼AB() ;
  ... ;
  }

Parcours infixe

```
1 Algorithme : infixe(r)
2 si r ≠ null alors
3 |   infixe(r.fg) ;
4 |   affichage r.cle ;
5 |   infixe(r.fd) ;
6 fin
```

Parcours infixe

```
1 Algorithme : prefixe(r)
2 si r ≠ null alors
3 |   affichage r.cle ;
4 |   prefixe(r.fg) ;
5 |   prefixe(r.fd) ;
6 fin
```

Parcours postfixe

```
1 Algorithme : postfixe(r)
2 si r ≠ null alors
3 |   postfixe(r.fg) ;
4 |   postfixe(r.fd) ;
5 |   affichage r.cle ;
6 fin
```

Hauteur

Définition

On appelle **hauteur** h, ou **profondeur**, d'un noeud x d'un arbre binaire B de racine r, la fonction définie récursivement comme suit :

$$\begin{aligned} h(r) &= 0 \\ h(x) &= 1 + h(y), \quad \text{si } y \text{ est le père de } x \end{aligned}$$

□

Définition

On appelle hauteur h, ou profondeur, d'un arbre binaire B, la valeur

$$h(B) = max\{h(x) \mid x \; est \; un \; noeud \; de \; B\}$$

□

Propriétés de la hauteur

Définition

Un arbre binaire B dont tous les noeuds (autre que les feuilles) sont doubles est dit **complet**.

Théorème

Un arbre binaire B est complet ssi il a 2^i noeuds de hauteur i pour i allant de 0 à $h(B)$. Le nombre total de ses noeuds est $2^{h(B)+1} - 1$ $\square$

Théorème

Soit B un arbre binaire de n noeuds et de hauteur $h(B)$. On a

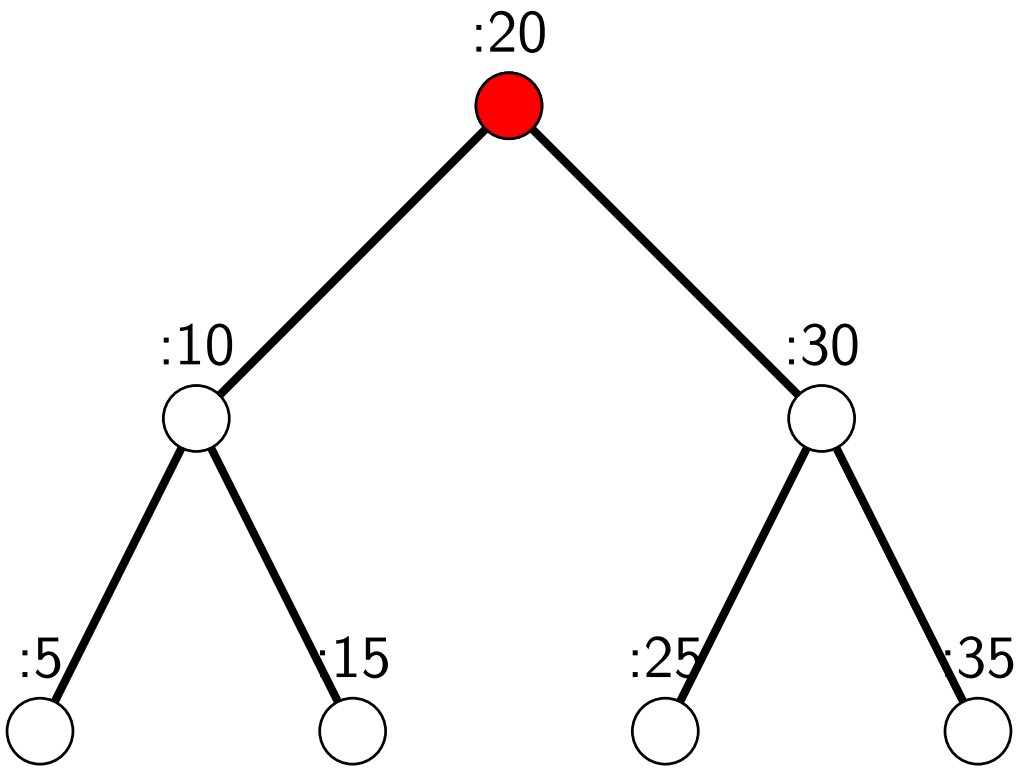
$$log_2(n) \leq h(B) \leq n - 1$$

Définitions

Définition

Un Arbre Binaire de Recherche (ABR) est un arbre binaire dont la valeur de la clé de chaque noeud est plus grande, strictement, que celle des noeuds de son sous-arbre de gauche et plus petite, ou égale, que celles des noeuds du sous-arbre droit. □

Exemple ABR



Contre-exemple ABR

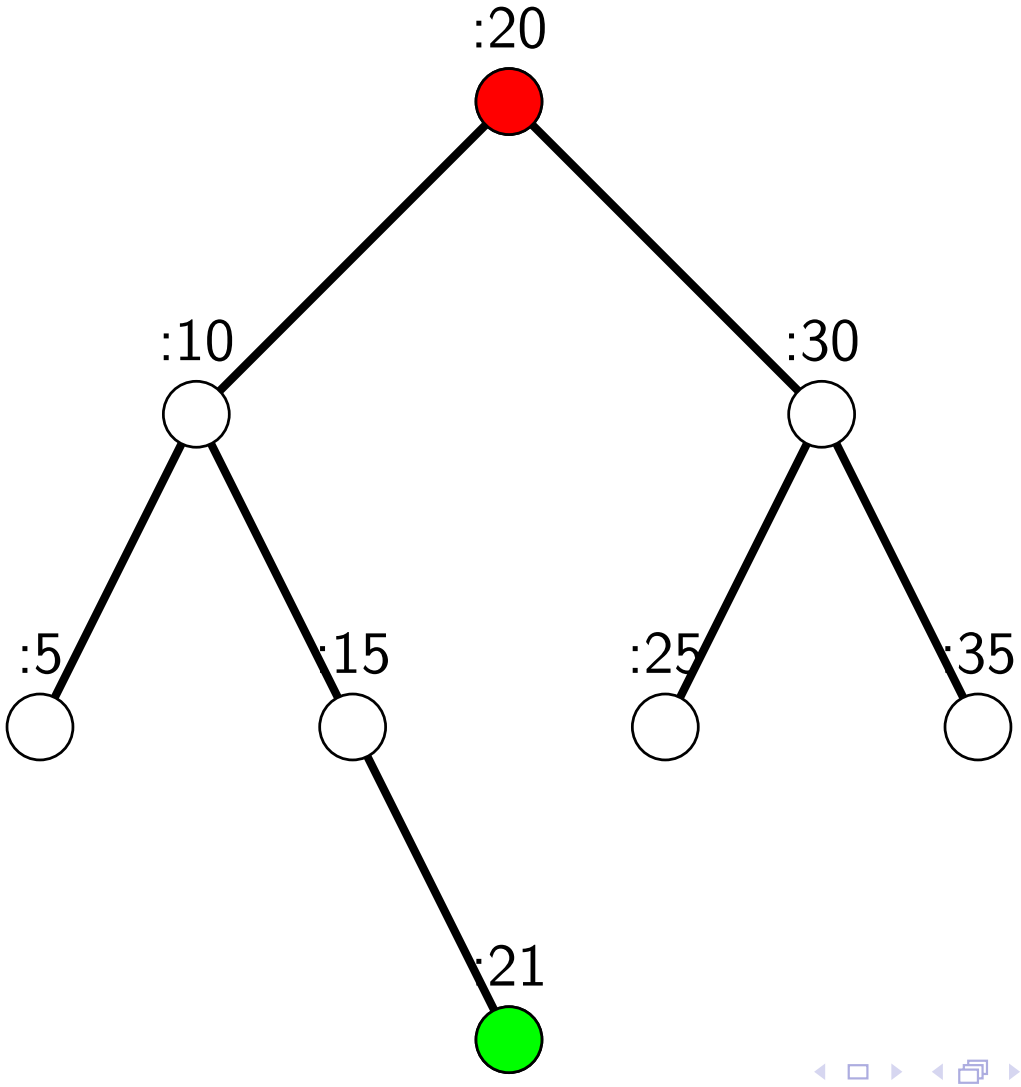


Illustration avec la librairie STL

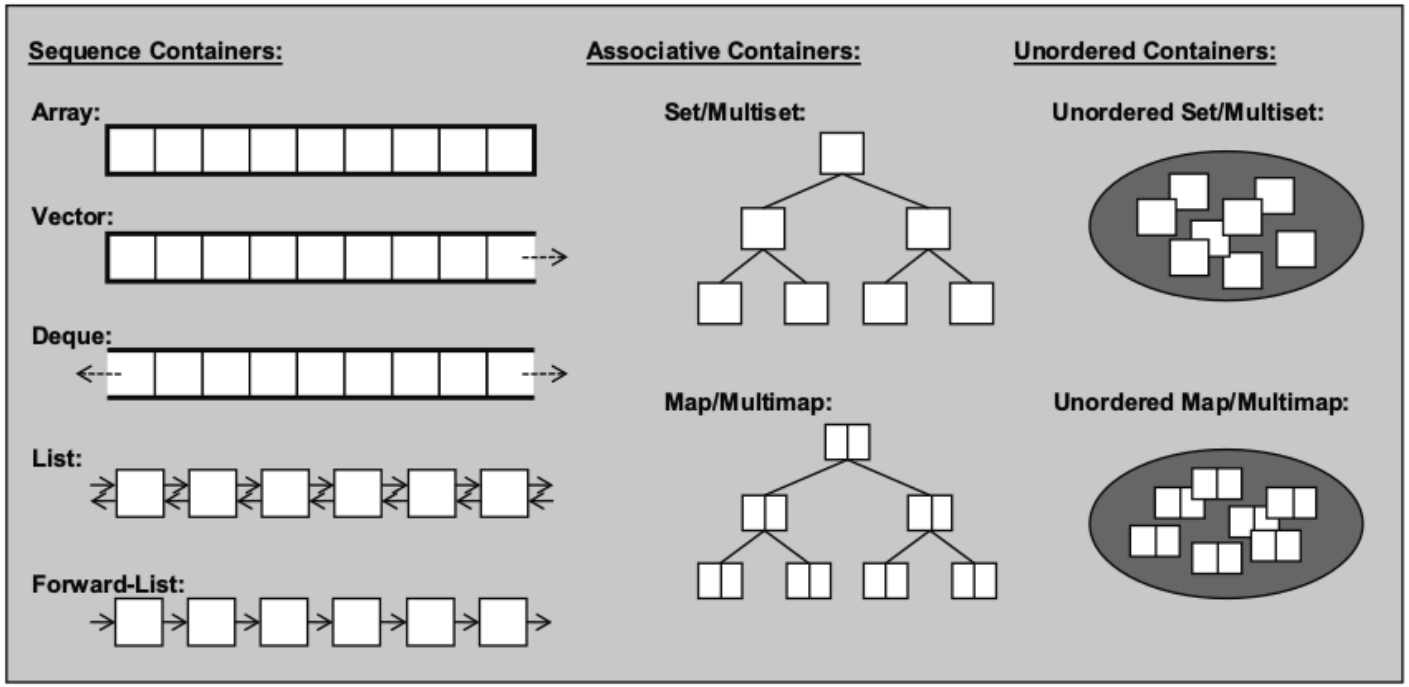


Figure 6.2. STL Container Types

from the C++ Standard Library - A Tutorial and Reference. 2nd ed - Nicolai M. Josuttis(Pearson, 2012)

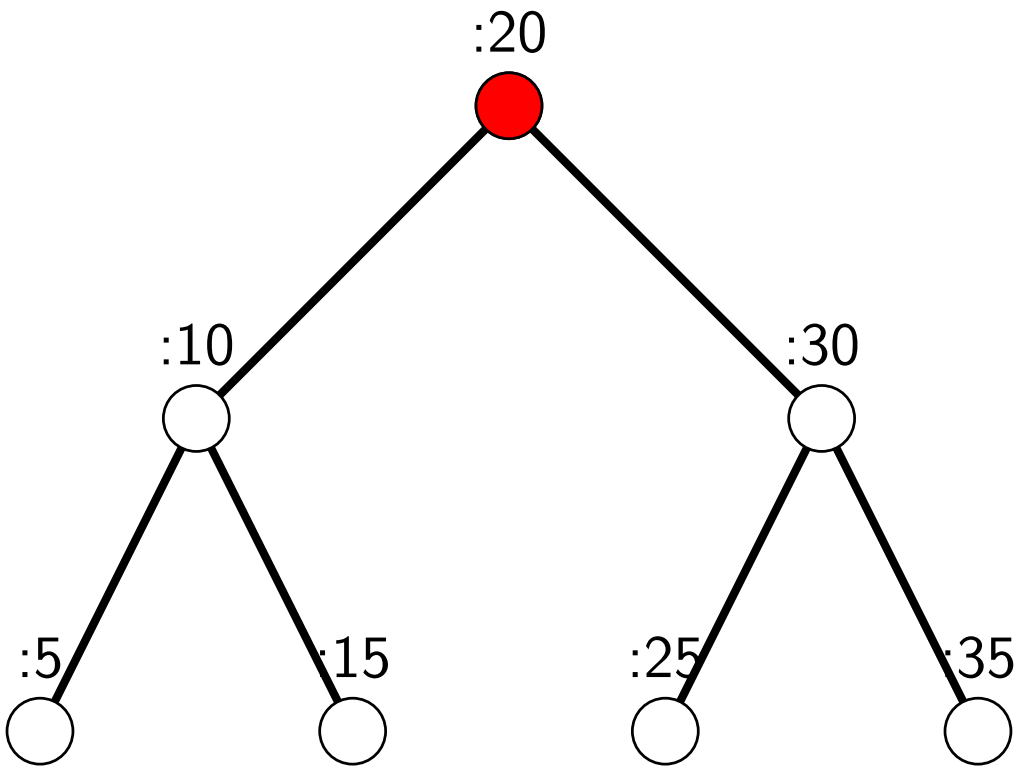
Recherche récursive dans un ABR

```
1  Algorithme : rechercher(r,k)
2  si (r == null) ou (k == r.cle) alors
3  |   retourner(r);
4  fin
5  si (k < r.cle) alors
6  |   retourner(rechercher(r.fg,k));
7  fin
8  sinon
9  |   retourner(rechercher(r.fd,k));
10 fin
```

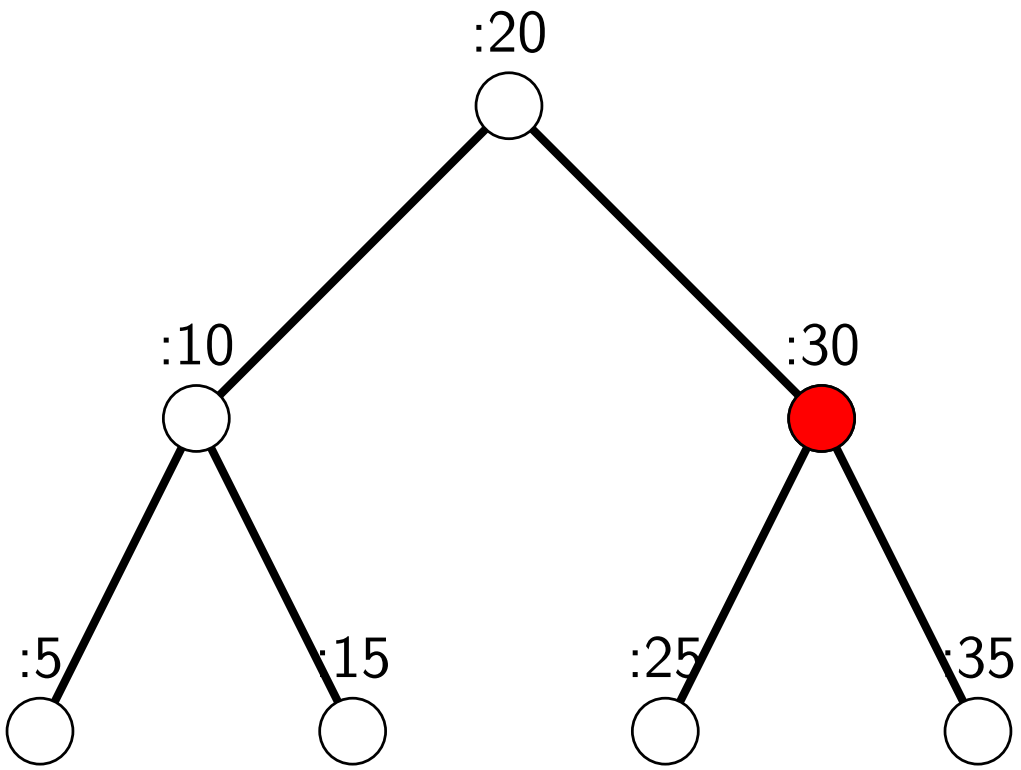
Recherche itérative dans un ABR

```
1  Algorithme : rechercher(r, int k)
2  tant que (r ≠ null) et (k ≠ r.cle)) faire
3  |   si (k < r.cle) alors
4  |   |   r = r.fg ;
5  |   fin
6  |   sinon
7  |   |   r = r.fd ;
8  |   fin
9  fin
10 retourner(r);
```

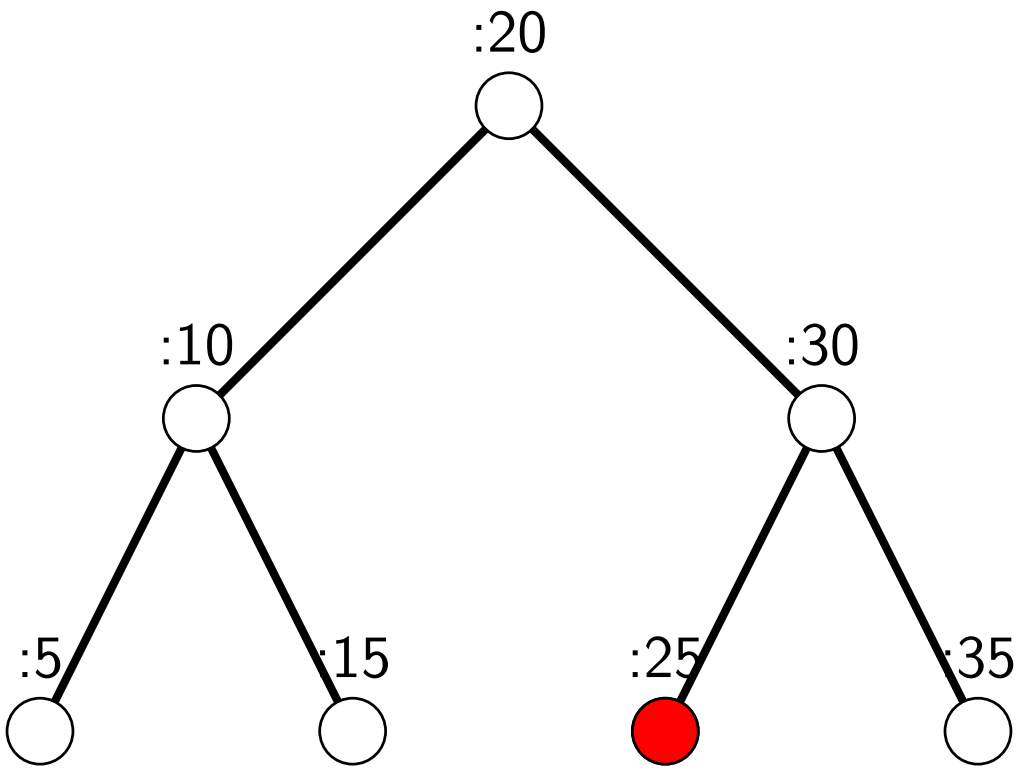
Exemple recherche (On cherche 25)



Exemple recherche (On cherche 25)



Exemple recherche (On cherche 25)



Affichage d'un ABR

Remarque

L'affichage infixe d'un ABR donne les clés triées en ordre croissant (ou décroissant)

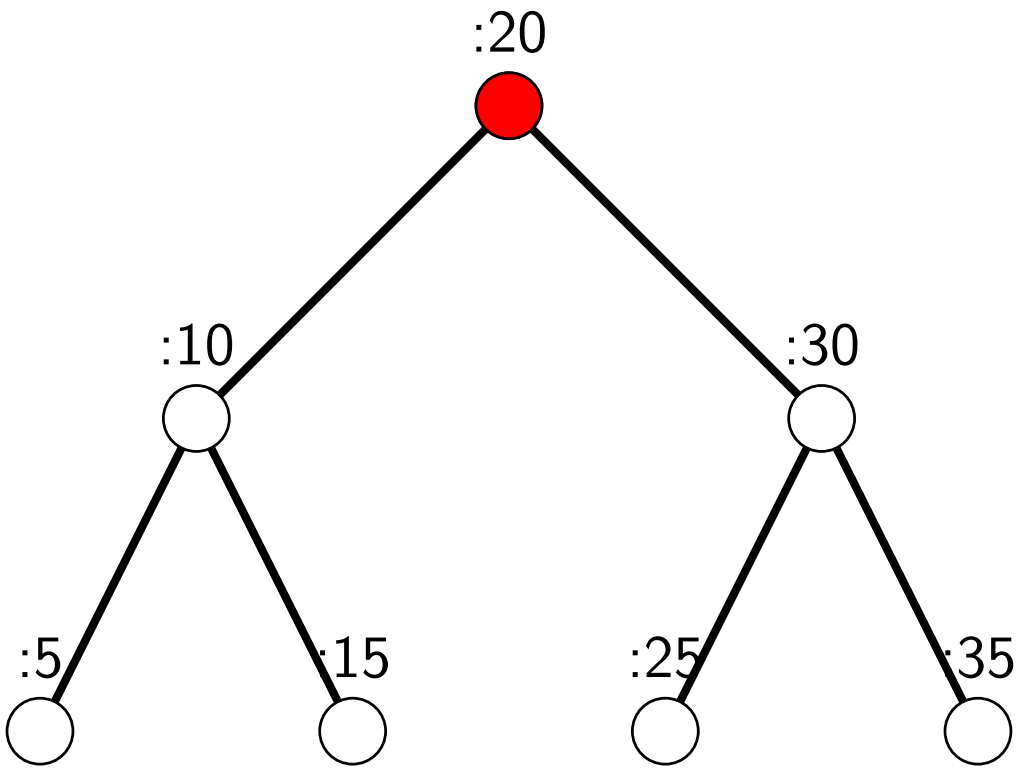
Cette propriéte est utilisée pour :

- reconnaitre un ABR,
- tester l'équivalence de deux ABR.
- savoir si un ABR est inclus dans un autre.

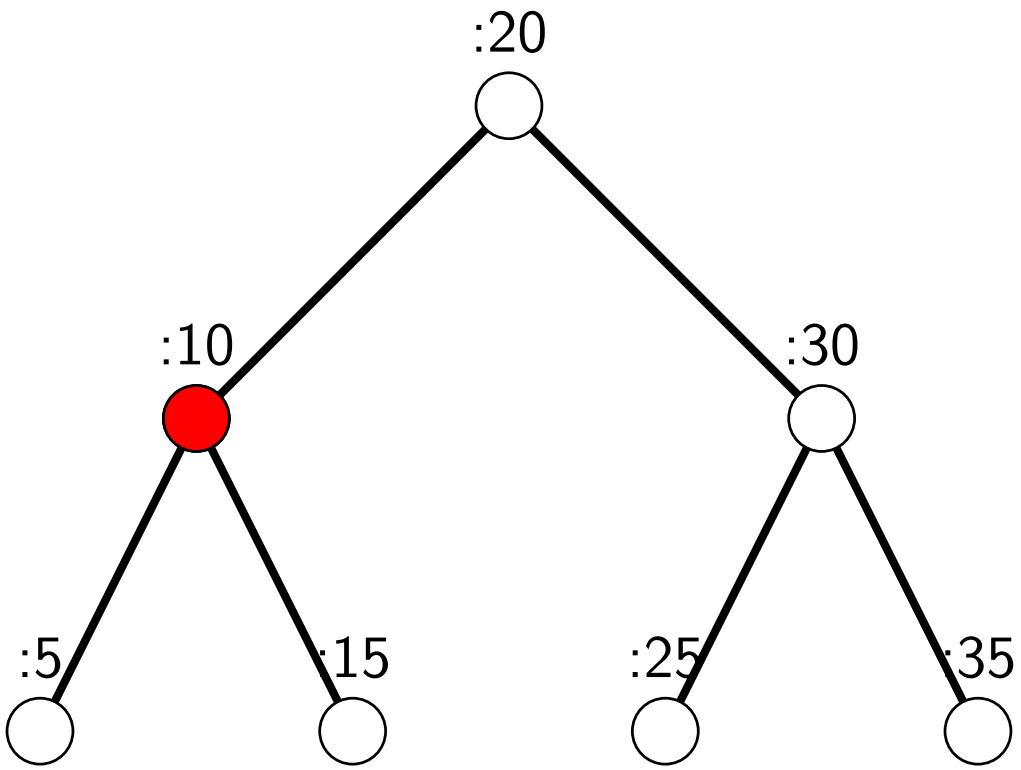
Insertion en feuille dans un ABR

```
1  Algorithme : insertionFeuille(r,y)
2  x = r, pere = null ;
3  tant que (x ≠ null) faire
4  |     pere = x ;
5  |     si (y.cle ≤ x.cle) alors
6  |     |     x = x.fg ;
7  |     fin
8  |     sinon
9  |     |     x = x.fd ;
10 |     fin
11 fin
12 y.pere = pere ;
13 si (y.pere == null) alors
14 |     r = y ;
15 fin
16 sinon
17 |     si (y.cle ≤ y.pere.cle) alors
18 |     |     y.pere.fg = y ;
19 |     fin
20 |     sinon
21 |     |     y.pere.fd = y ;
22 |     fin
23 fin
```

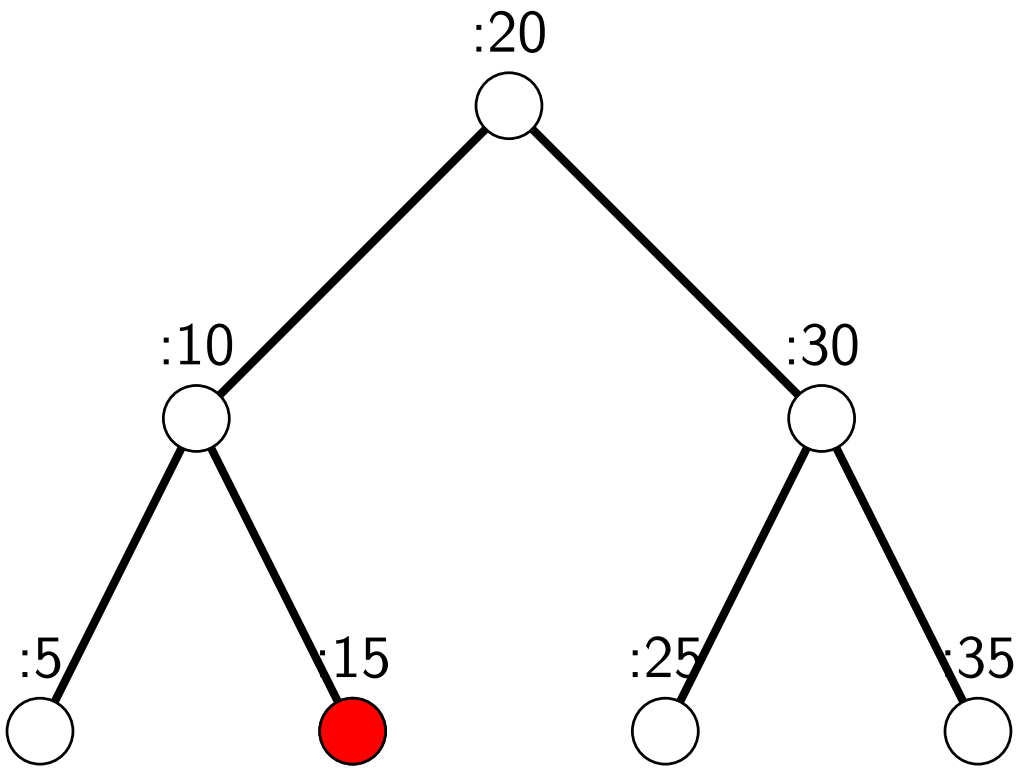
Exemple Insertion (de 12)



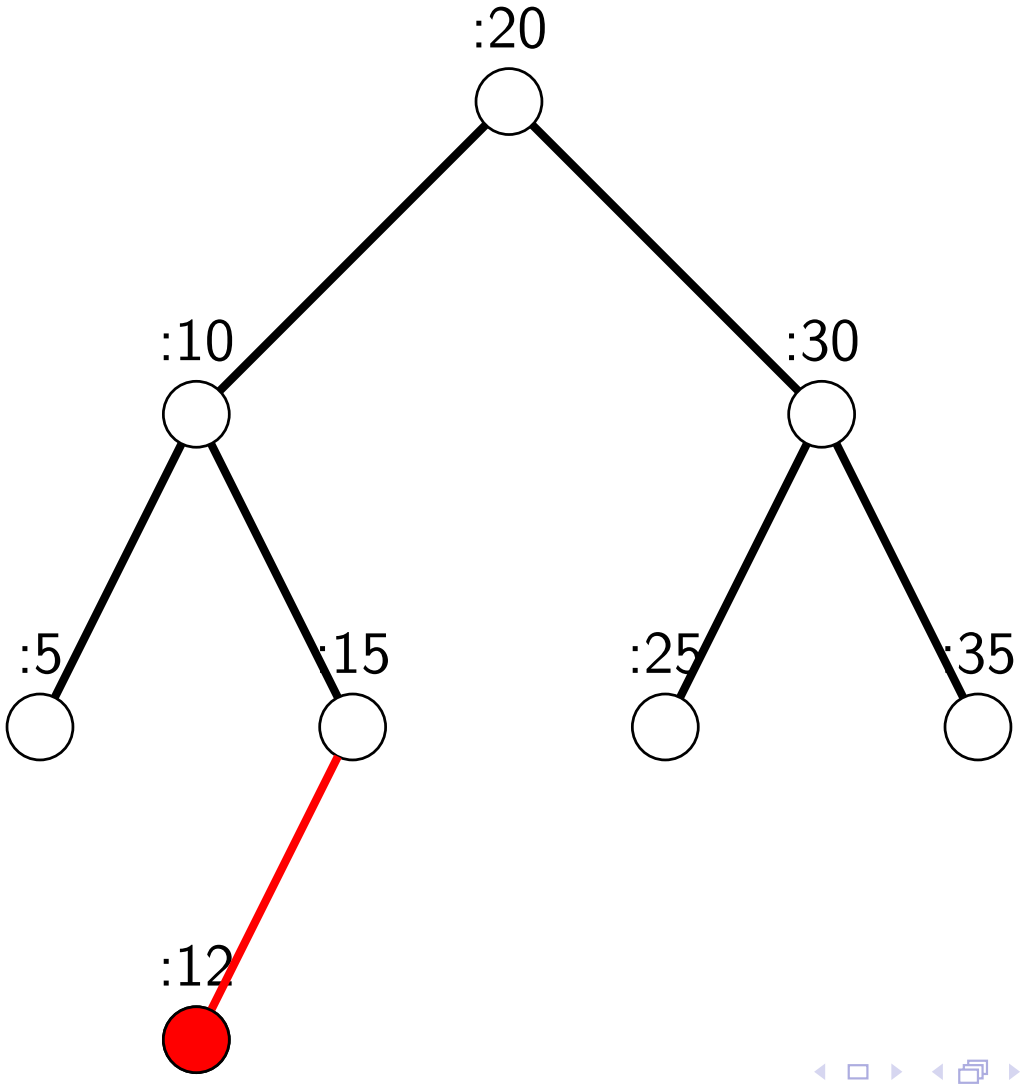
Exemple Insertion (de 12)



Exemple Insertion (de 12)



Exemple Insertion (de 12)

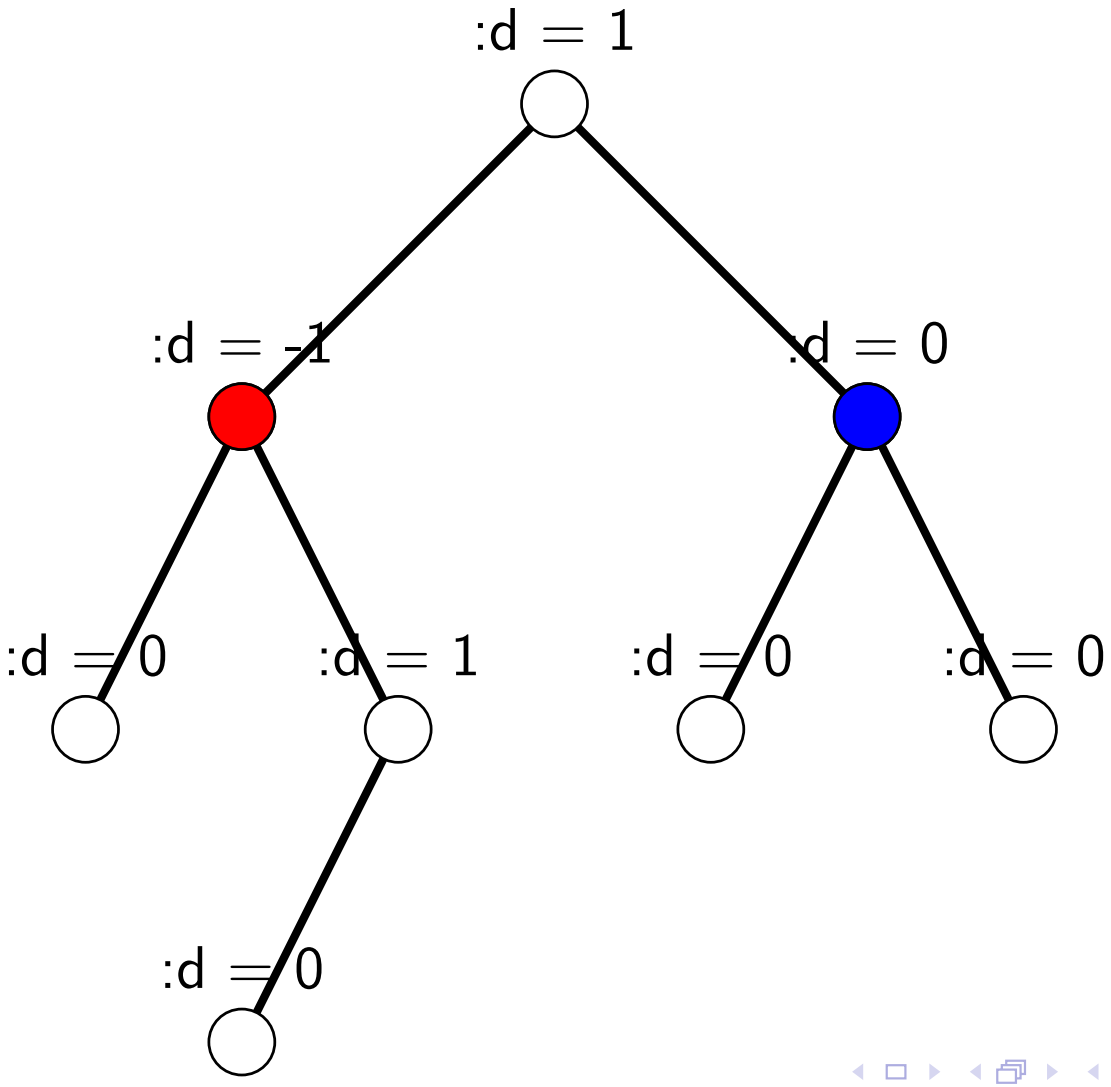


Arbres équilibrés de type AVL (Adelson - Velskii - Landis)

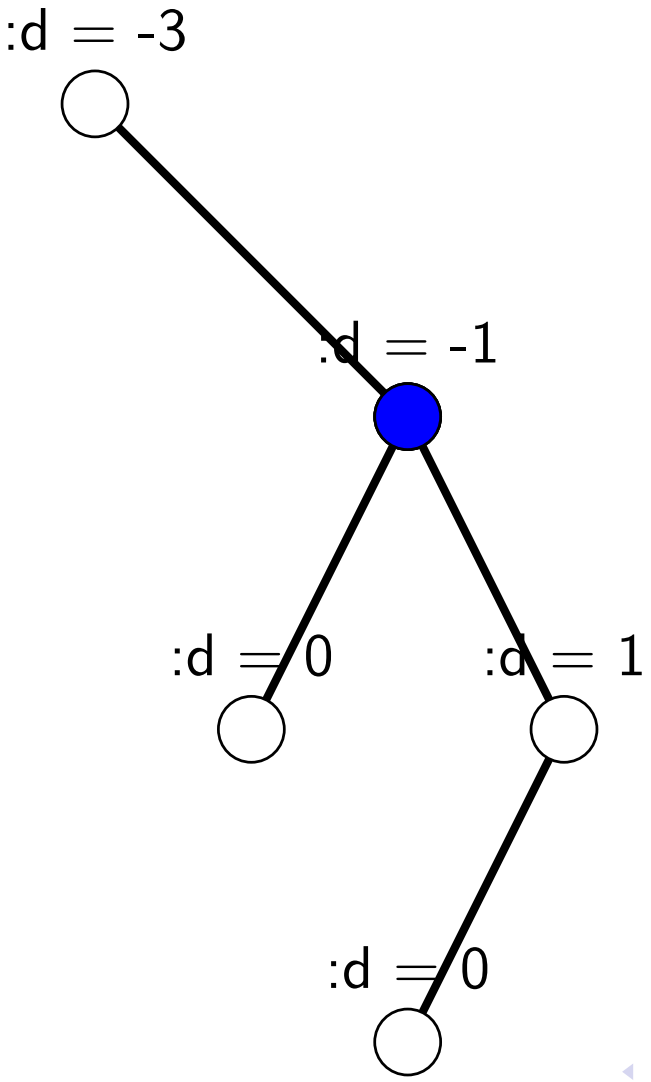
Définition

Soit T un arbre binaire de racine r, on appelle **déséquilibre** de T la différence $d(r) = h(G) - h(D)$ où $h(G)$ (resp. $h(D)$) désigne la hauteur du sous-arbre de racine $r.fg$ (resp. $r.fd$).

Exemple



Exemple



Arbres équilibrés de type AVL (Adelson - Velskii - Landis)

Définition

T est dit **équilibré (resp. déséquilibré)** si seulement si $|d(v)| \leq 1$ (resp $|d(v)| \geq 2$) en tout noeud v de T.

Définition

Un AVL est un arbre binaire de recherche équilibré.

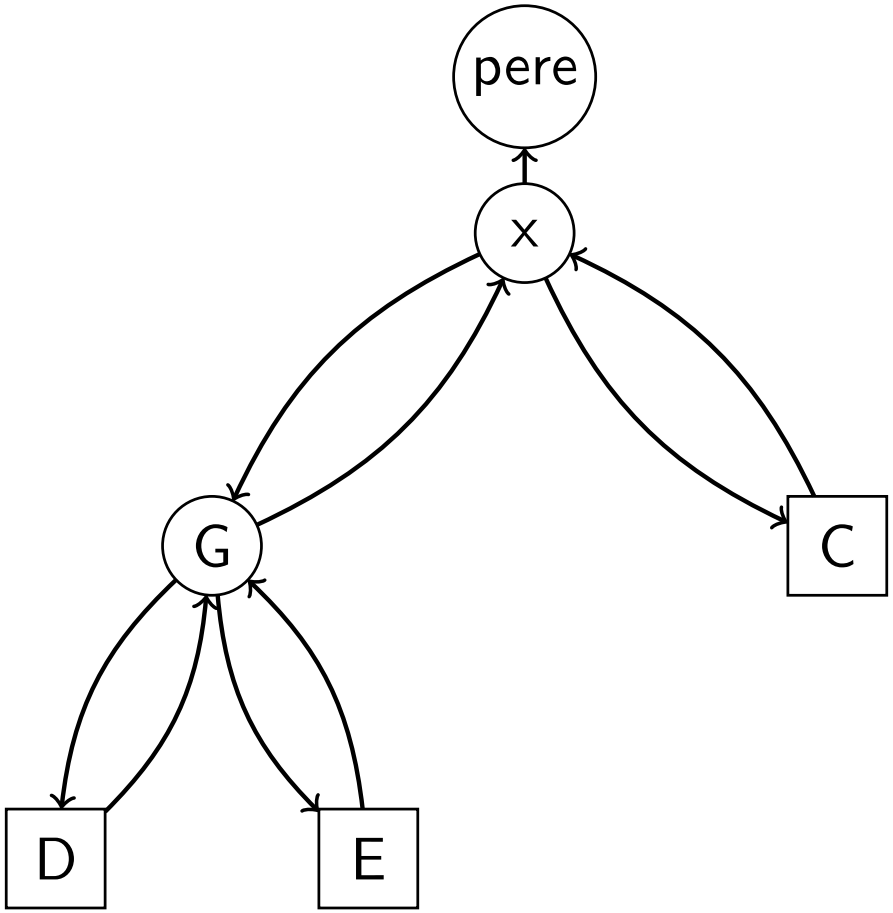
Opérations de rotation

Elles visent à rééquilibrer un arbre si il ne l'est pas. Il y en a 4 fondamentales :

- rotation droite
- rotation gauche
- rotation gauche-droite
- rotation droite-gauche

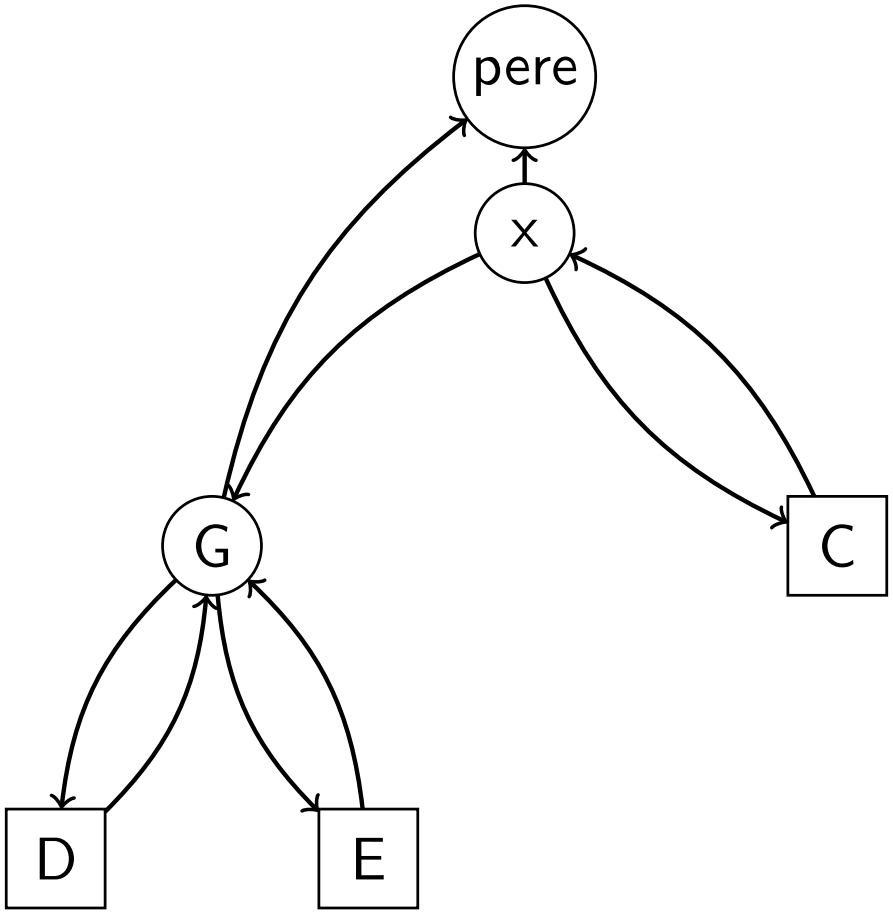
Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



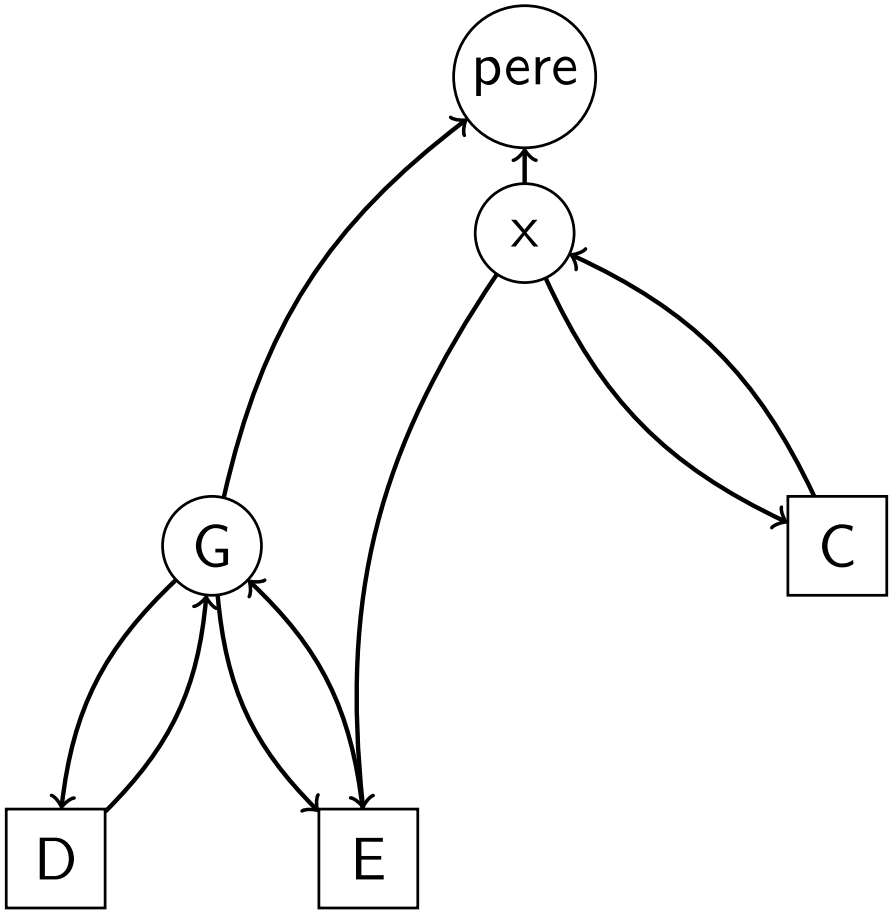
Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



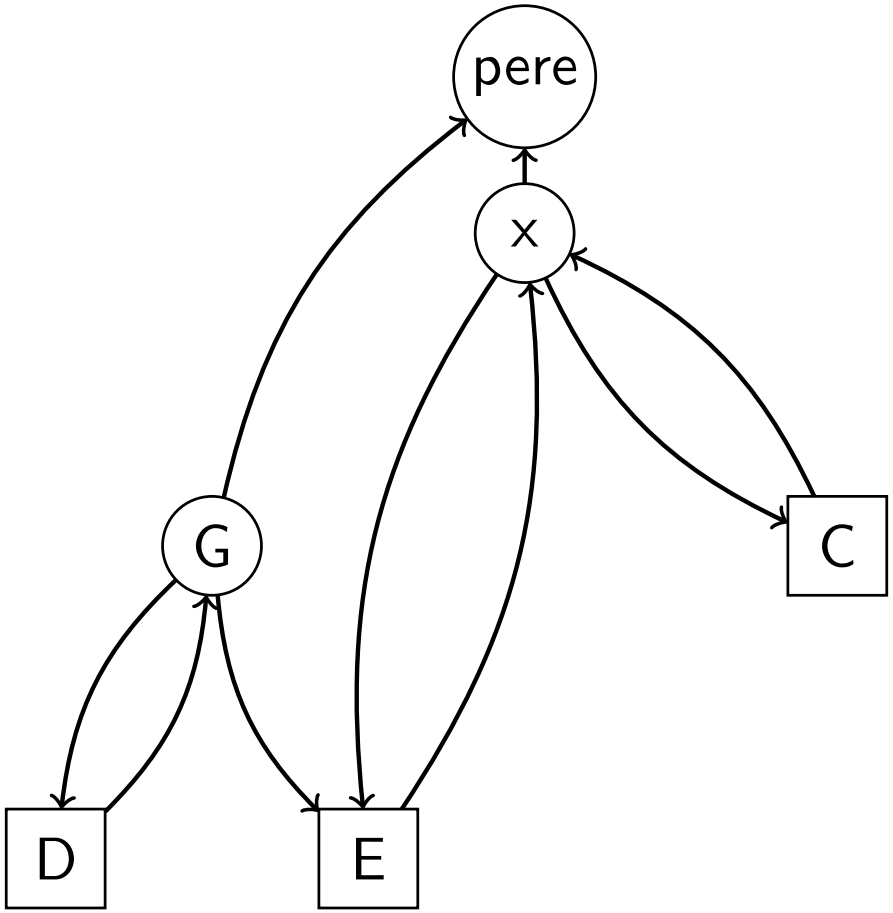
Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



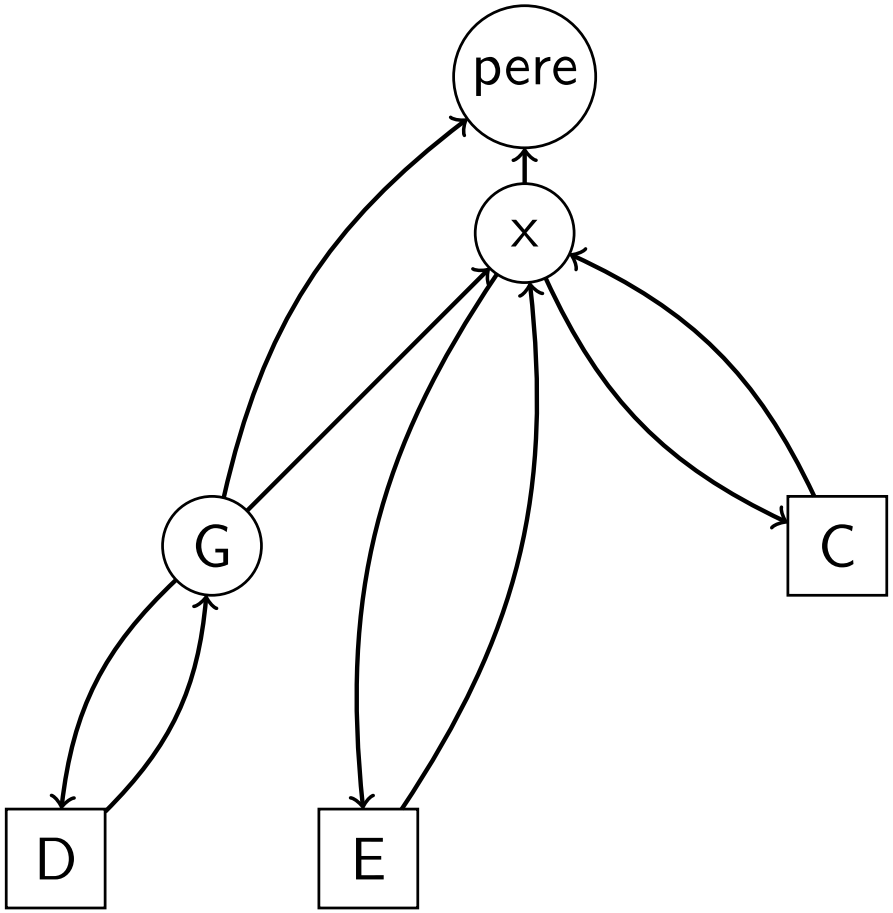
Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



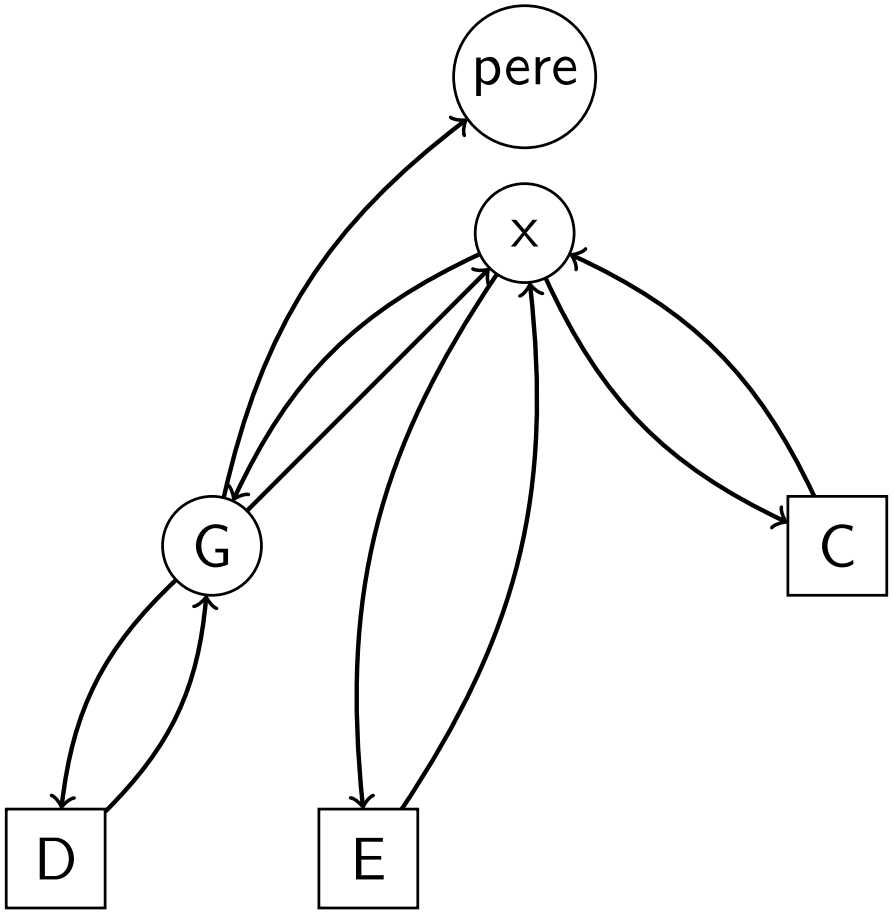
Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



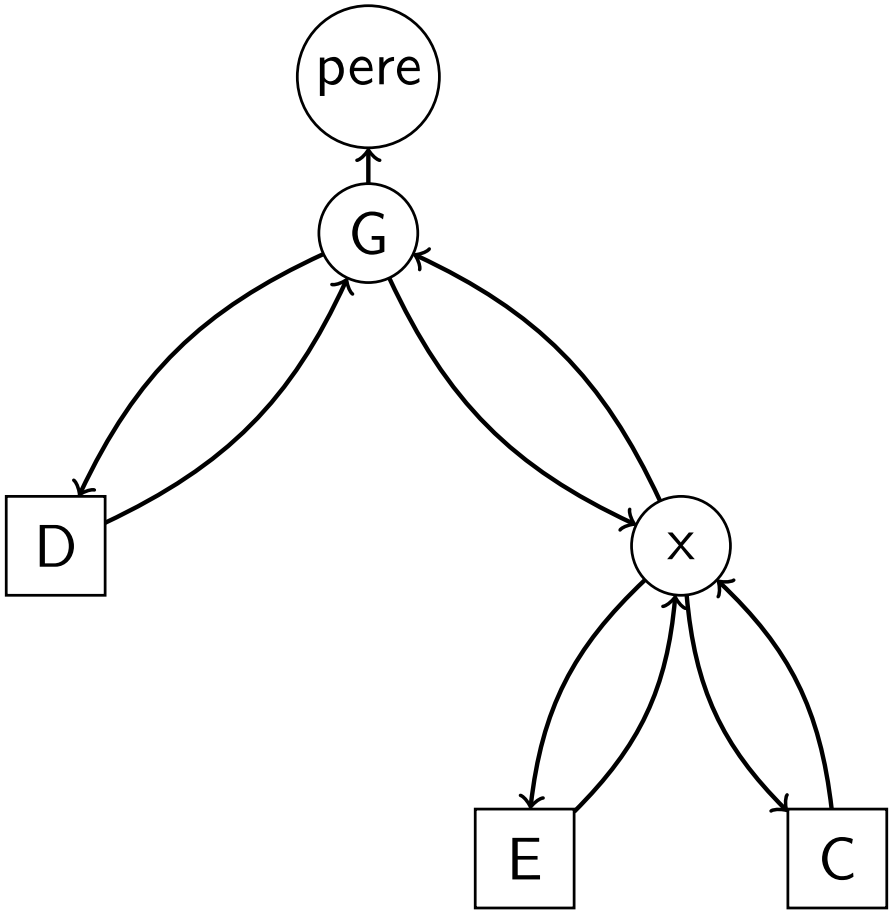
Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



Rotation droite

$d(x) = 2$ et $d(x.fg) = 1$



Rotation droite sur un noeud x se trouvant dans un arbre enraciné en r. x supposé non nul ici.

Algorithme : rotationDroite(r,x)

```
p = x.pere;
G = x.fg ;
G.pere = p;
x.fg = G.fd ;
si G.fd ≠ NULL alors
    G.fd.pere = x;
fin
G.fd = x ;
x.pere = G;
si p == NULL alors
    r = G ;
fin
sinon
    si p.fg == x alors
        p.fg = G;
    fin
    sinon
        p.fd = G;
    fin
fin
```

Rotation gauche

Si $d(x) = -2$ et $d(x.fd) = -1$ alors rotationGauche(x).

Symétrique de la rotation droite : Remplacer G par D et fg par fd partout.

Rotation gauche-droite

Si $d(x) = 2$ et $d(x.fg) = -1$

```
1 Algorithme : rotationGaucheDroite(x)
2 rotationGauche(x.fg) ;
3 rotationDroite(x) ;
```

Rotation droite-gauche

Si $d(x) = -2$ et $d(x.fd) = 1$

```
1 Algorithme : rotationDroiteGauche(x)
2 rotationDroite(x.fd) ;
3 rotationGauche(x) ;
```

Insertion en feuille avec ré-équilibrage

```
1 Algorithme : insertionFeuilleEquilibre(r,y)
2 insertion(r,y) ;
3 si y.pere ≠ null alors
4 |   requilibre(y.pere) ;
5 fin
```

Insertion en feuille avec ré-équilibrage

- Un ré-équilibrage peut être fait par la méthode “requilibre(v)”.
- Tout noeud “v” y a deux autres attributs : “h” (hauteur de l’arbre enraciné en v), “d” déséquilibre de l’arbre enraciné en “v”.
- Les méthodes de rotation doivent mettre à jour “h” et “d”. Cela suppose des ajouts à la méthode donnée page 45 (à faire en TP).

Réquilibrage à partir d'un noeud v

```
Algorithme : requilibre(v)
anc_h = v.h
si v.fg == null alors
  |   hg = -1 ; dg = 0;
fin
sinon
  |   hg = v.fg.h ; dg = v.fg.d;
fin
si v.fd == null alors
  |   hd = -1 ; dd = 0;
fin
sinon
  |   hd = v.fd.h ; dd = v.fd.d;
fin
v.d = hg − hd ; v.h = max(hg, hd) + 1 ;
si (v.d == 2) et (dg == 1) alors
  |   rotationDroite(v);
fin
sinon si (v.d == −2) et (dd == −1) alors
  |   rotationGauche(v);
fin
sinon si (v.d == 2) et (dg == −1) alors
  |   rotationGaucheDroite(v);
fin
sinon si (v.d == −2) et (dd == 1) alors
  |   rotationDroiteGauche(v);
fin
si anc_h ≠ v.h et v.pere ≠ null alors
  |   requilibre(v.pere);
```

Hauteur d'un arbre équilibré

Théorème

Tout arbre équilibré de n noeuds a une hauteur h vérifiant :

$$log_2(n+1) \leq h+1 \leq 1.44.log_2(n+2)$$

Preuve

• L'AVL de hauteur h comportant le plus noeuds est un arbre complet. Son nombre de noeuds est $2^{h+1} - 1$. On a donc

$$n \leq 2^{h+1} - 1,$$
$$n + 1 \leq 2^{h+1},$$
$$log_2(n+1) \leq h + 1.$$

• L'AVL de hauteur h comportant le moins noeuds est un arbre dans lequel le déséquilibre de chaque noeud **interne** est égal à 1 ou -1. Soit $N(h)$ le nombre de noeuds minimal pour un tel AVL. Ses fils de hauteurs $h-1$ et $h-2$ comportent également le moins de noeuds possibles. On a donc la relation de récurrence suivante :

$$N(h) = 1 + N(h-1) + N(h-2)\ ,\ h \geq 2$$

avec $N(0) = 1$ et $N(1) = 2$.
On en déduit que :

$$N(h) + 1 = 2 + N(h-1) + N(h-2) = (N(h-1) + 1) + (N(h-2) + 1).$$

En posant $F(h) = N(h) + 1$, on obtient la suite de Fibonacci suivante :

$$F(h) = F(h-1) + F(h-2)\ ,\ h \geq 2.$$

avec $F(0) = 2$ et $F(1) = 3$.

Preuve (suite)

Nous savons (voir outils mathématiques) que $F(h)$ peut s'écrire :

$$F(h) = \frac{1}{\sqrt{5}}(r_1^{h+3} - r_2^{h+3}),$$

où $r_1 = \frac{1+\sqrt{5}}{2}$ et $r_2 = \frac{1-\sqrt{5}}{2}$.
Donc :

$$n + 1 \geq N(h) + 1 = F(h) = \frac{1}{\sqrt{5}}(r_1^{h+3} - r_2^{h+3}),$$

$$n + 1 \geq \frac{1}{\sqrt{5}}(r_1^{h+3} - 1),\ car \left|r_2^{h+3}\right| < 1$$

$$\sqrt{5}(n+1) + 1 \geq r_1^{h+3},$$

$$log_{r_1}(\sqrt{5}(n+1) + 1) \geq h + 3.$$

Comme $\sqrt{5} > 1$, et log est une fonction croissante on a

$$log_{r_1}(\sqrt{5}(n+2)) > log_{r_1}(\sqrt{5}(n+1) + 1) \geq h + 3,$$

or $log_{r_1}(\sqrt{5}(n+2)) = log_{r_1}(\sqrt{5}) + log_{r_1}(n+2)$.

Preuve (suite)

Donc $log_{r_1}(\sqrt{5}(n+2)) = log_{r_1}(\sqrt{5}) + \frac{log_2(n+2)}{log_2(r_1)}$. D'autre part, $log_{r_1}(\sqrt{5}) < 2$. On en déduit donc que :

$$2 + \frac{log_2(n+2)}{log_2(r_1)} \geq h + 3,$$

$$\frac{log_2(n+2)}{log_2(r_1)} \geq h + 1.$$

Tenant compte du fait que $\frac{1}{log_2(r_1)} \simeq 1.44$, on a finalement :

$$1.44.log_2(n+2) \geq h + 1.$$

Notion d'arbre binaire partiellement ordonné

Définition

Un arbre binaire partiellement ordonné est un arbre dont chaque noeud est étiquetté par une valeur telle que la valeur d'un noeud est toujours inférieure ou égale à la valeur de chacun de ses fils.

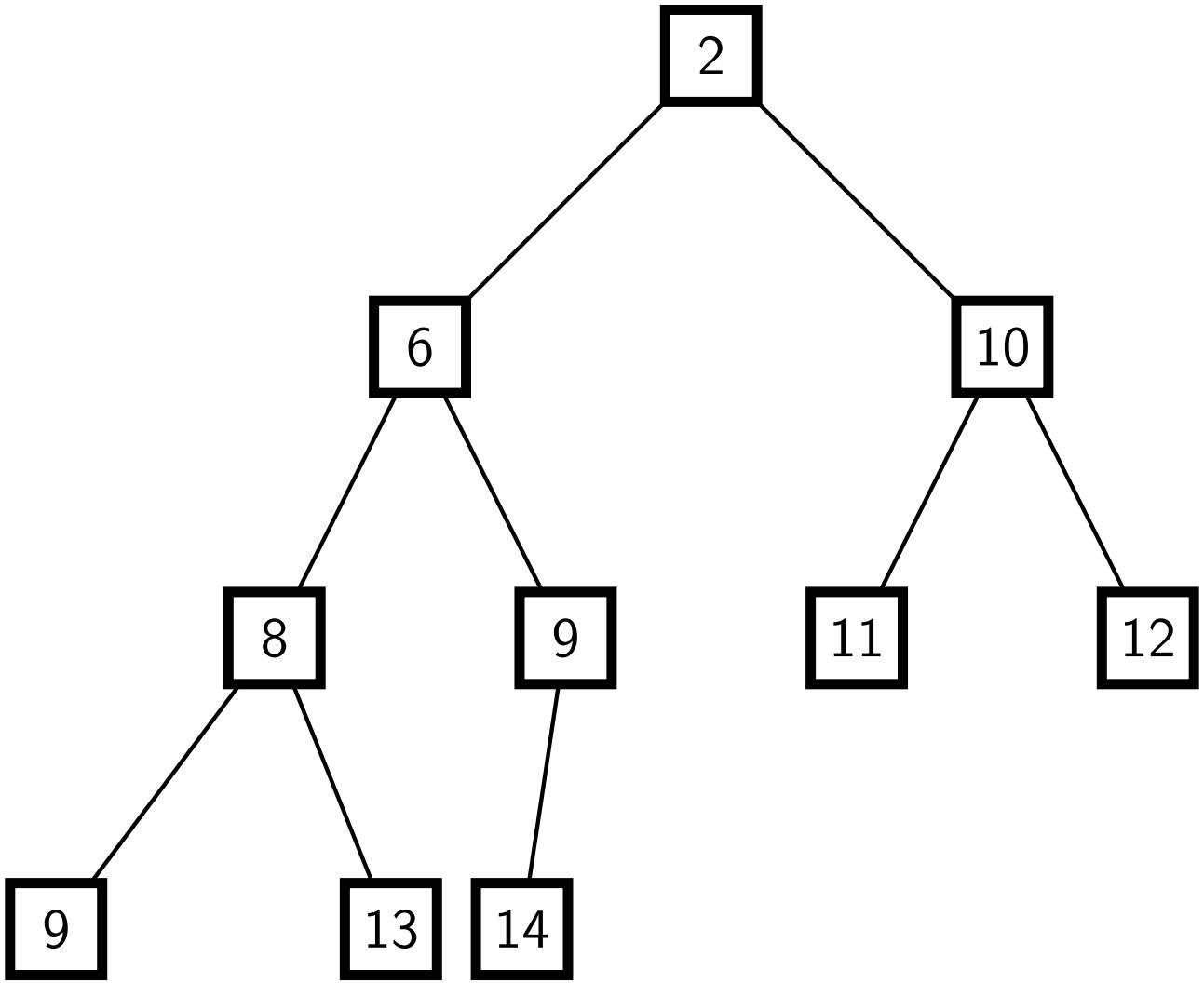
Définition

Un arbre binaire est dit **parfait** ssi tous ses niveaux sont complets sauf éventuellement le dernier et dans ce cas les feuilles sont groupées le plus à gauche possible.

Définition

Un tas est un tableau représentant un arbre binaire parfait partiellement ordonné.

Exemple



Exemple

T =

1	2	3	4	5	6	7	8	9	10
2	6	10	8	9	11	12	9	13	14

Propriétés de T

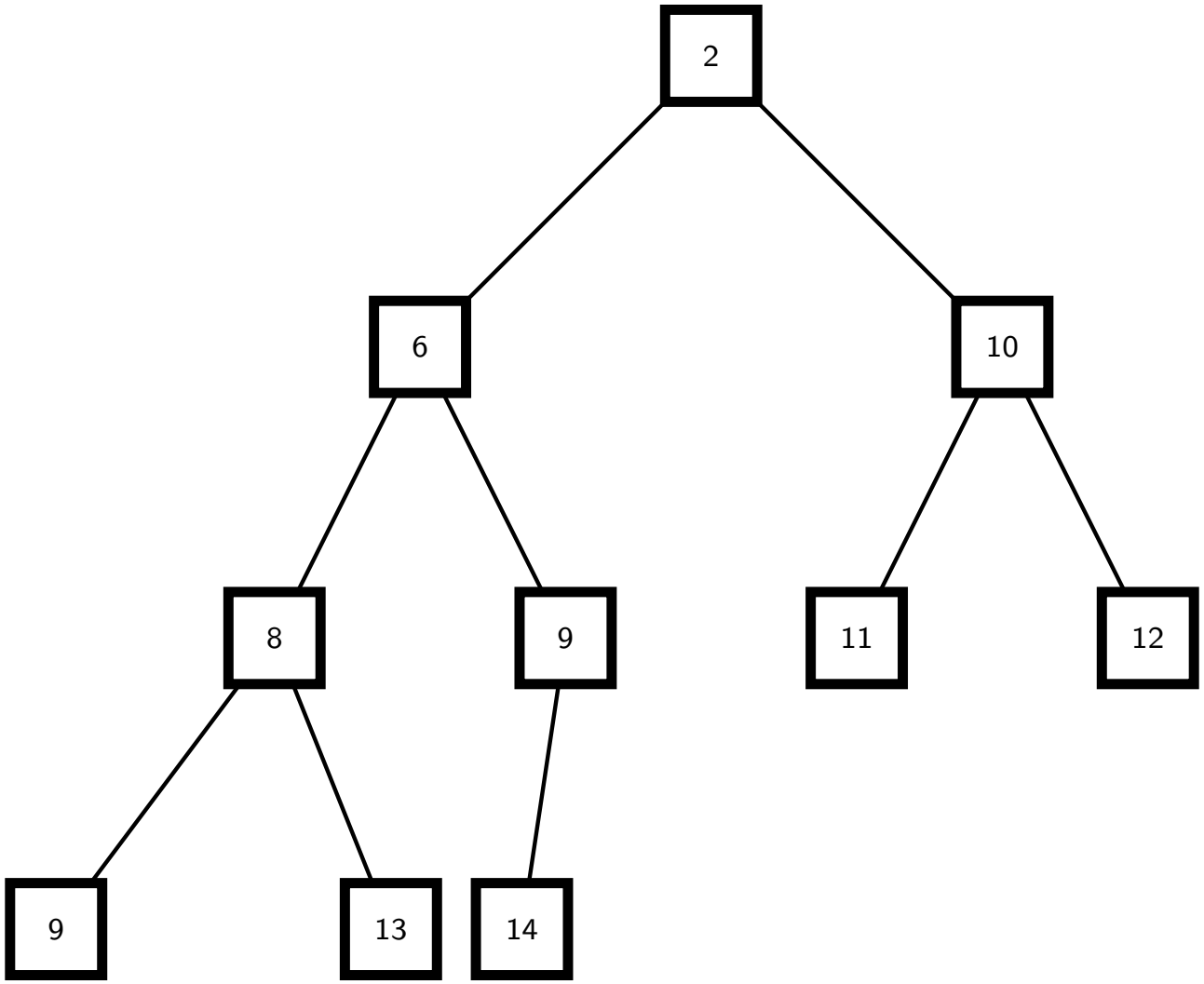
On suppose l'existence de la case $T[n+1] = +\infty$.

Théorème

Soit n le nombre de noeuds du tas T.

- *Pour tout $i = 1, 2, ..., \lfloor \frac{n}{2} \rfloor$ on a :*
  - *le fils gauche du noeud d'indice i est le noeud $T[2i]$,*
  - *le fils droit du noeud d'indice i est le noeud $T[2i+1]$,*
- *Pour tout $i = 2, 3, ..., , n$ on a :*
  - *le père du noeud d'indice i est le noeud $T[\lfloor \frac{i}{2} \rfloor]$,*

Ajout de noeuds dans un tas

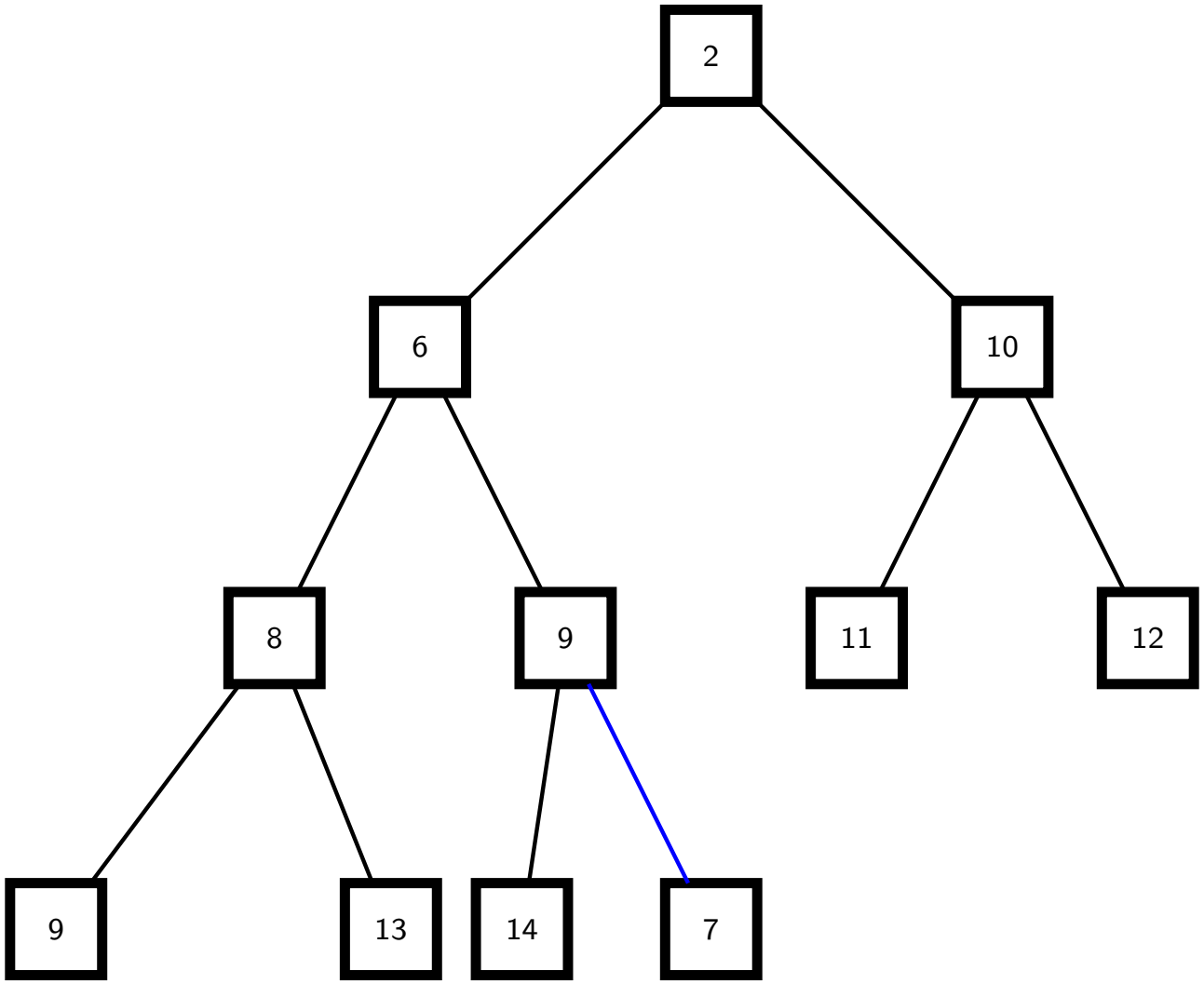


Ajout de noeuds dans un tas

T =

1	2	3	4	5	6	7	8	9	10
2	6	10	8	9	11	12	9	13	14

Ajout de noeuds dans un tas

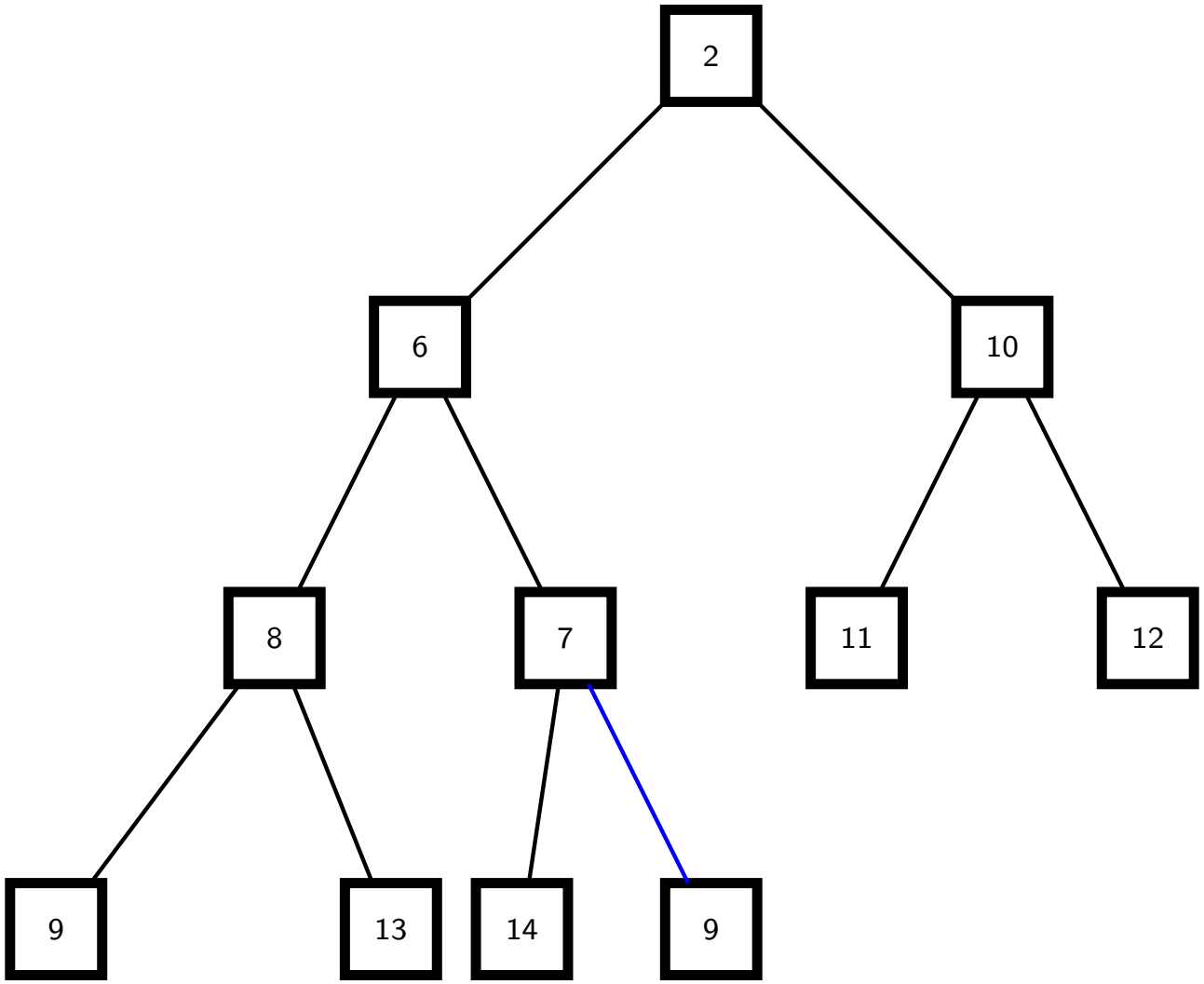


Ajout de noeuds dans un tas

T =

1	2	3	4	5	6	7	8	9	10	11
2	6	10	8	9	11	12	9	13	14	7

Ajout de noeuds dans un tas



Ajout de noeuds dans un tas

T =

1	2	3	4	5	6	7	8	9	10	11
2	6	10	8	7	11	12	9	13	14	9

Algorithme d'insertion dans un tas

Algorithme : Insertion(x,T,n)

$n = n + 1$;

$T[n] = x$;

$i = n$;

tant que $i \geq 1$ *et* $T[i] < T[\lfloor \frac{i}{2} \rfloor]$ **faire**

  $echanger(T[i], T[\lfloor \frac{i}{2} \rfloor])$;

  $i = \lfloor \frac{i}{2} \rfloor$

fin

Dénombrement du nombre d'échanges

Algorithme : Insertion(x,T,n)

$n = n + 1$;

$T[n] = x$;

$i = n$;

tant que $i \geq 1$ *et* $T[i] < T[\lfloor \frac{i}{2} \rfloor]$ **faire**

  $echanger(T[i], T[\lfloor \frac{i}{2} \rfloor])$;

  $i = \lfloor \frac{i}{2} \rfloor$

fin

Dénombrement du nombre d'échanges

Algorithme : Insertion(x,T,n)

$n = n + 1$;

$T[n] = x$;

$i = n$;

tant que $i \geq 1$ *et* $T[i] < T[\lfloor \frac{i}{2} \rfloor]$ **faire**

  $echanger(T[i], T[\lfloor \frac{i}{2} \rfloor])$;

  $i = \lfloor \frac{i}{2} \rfloor$

fin

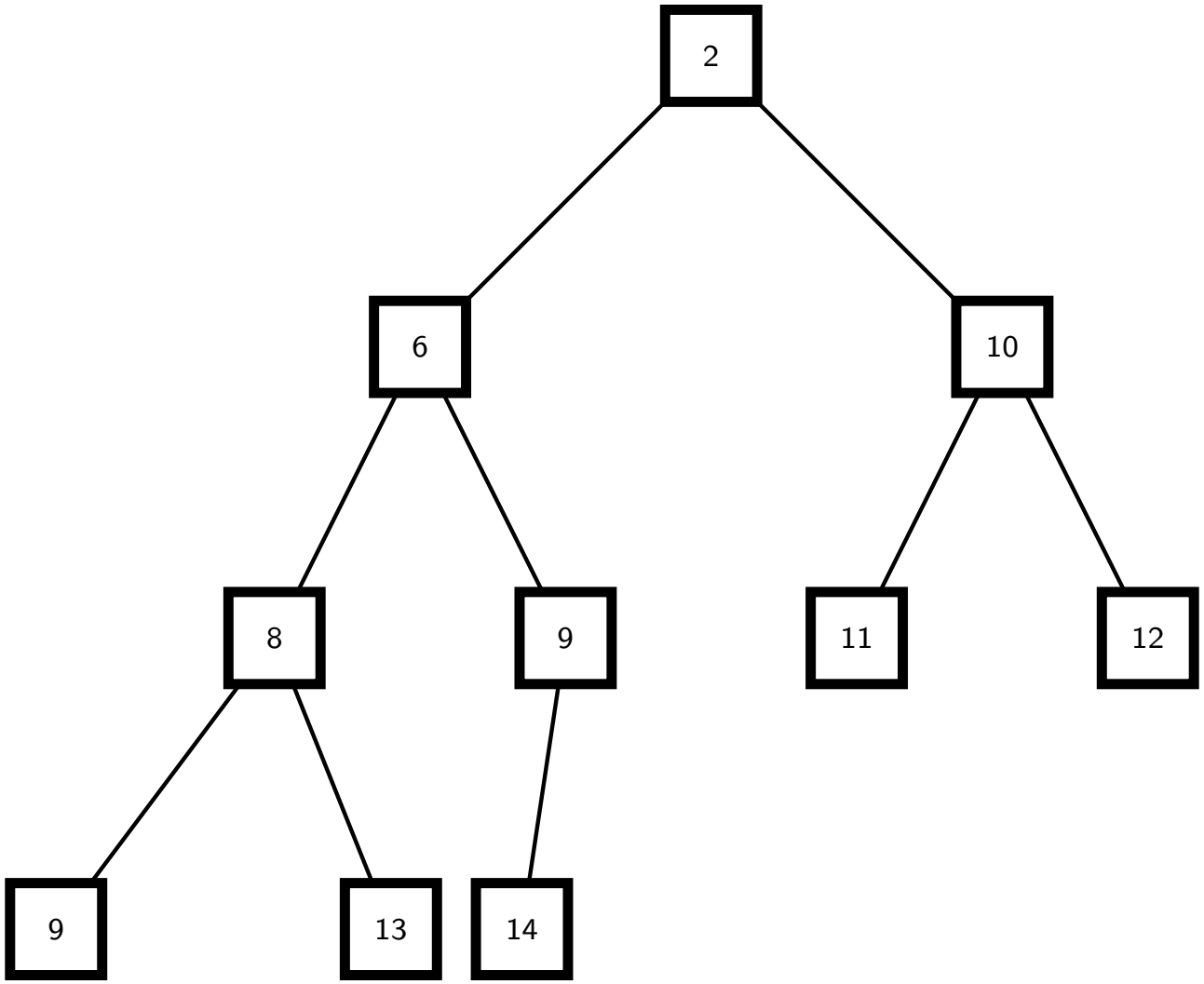
Nombre d'échanges = au pire la hauteur du tas.

Suppression dans un tas : principe

1. Soit r le noeud racine et $val(r)$ sa valeur (1ère case du tableau)
2. $Min = val(r)$
3. Remplacer $val(r)$ par la dernière valeur du tas (la dernière case du tableau)
4. $m = argmin\{val(gauche(r)), val(droite(r))\}$
5. Tant que $(val(r) > val(m))$
6. ..... Echanger dans l'arbre les valeurs des noeuds r et m
7. ..... r = m
8. ..... $m = argmin\{val(gauche(r)), val(droite(r))\}$
9. Fin Tant que
10. Renvoyer Min

On appellera "réorganisation" les instructions 4 à 9.

Suppression dans un tas : exemple

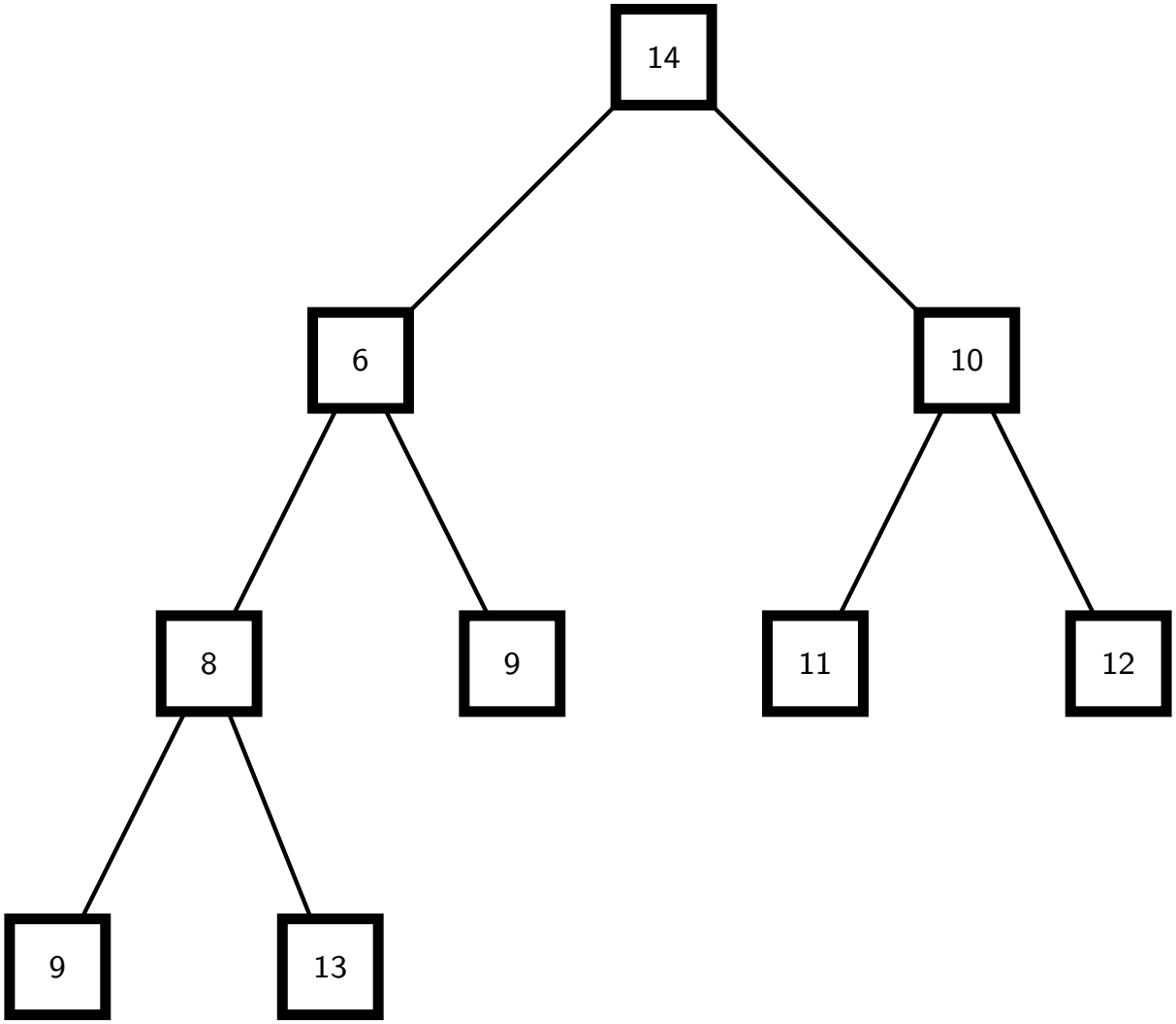


Suppression dans un tas : exemple

T =

1	2	3	4	5	6	7	8	9	10
2	6	10	8	9	11	12	9	13	14

Suppression dans un tas : exemple

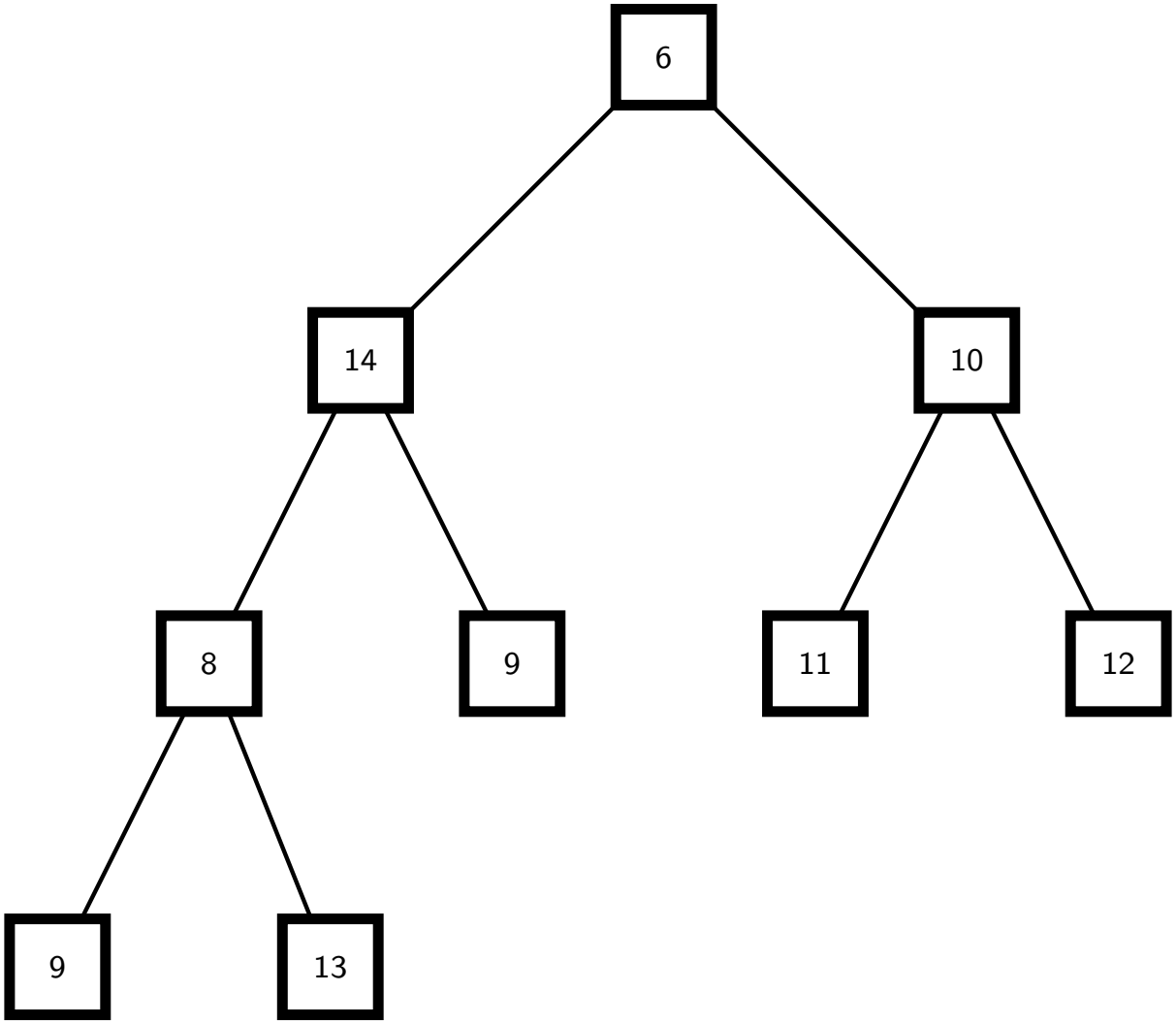


Suppression dans un tas : exemple

T =

1	2	3	4	5	6	7	8	9	10
14	6	10	8	9	11	12	9	13	14

Suppression dans un tas : exemple

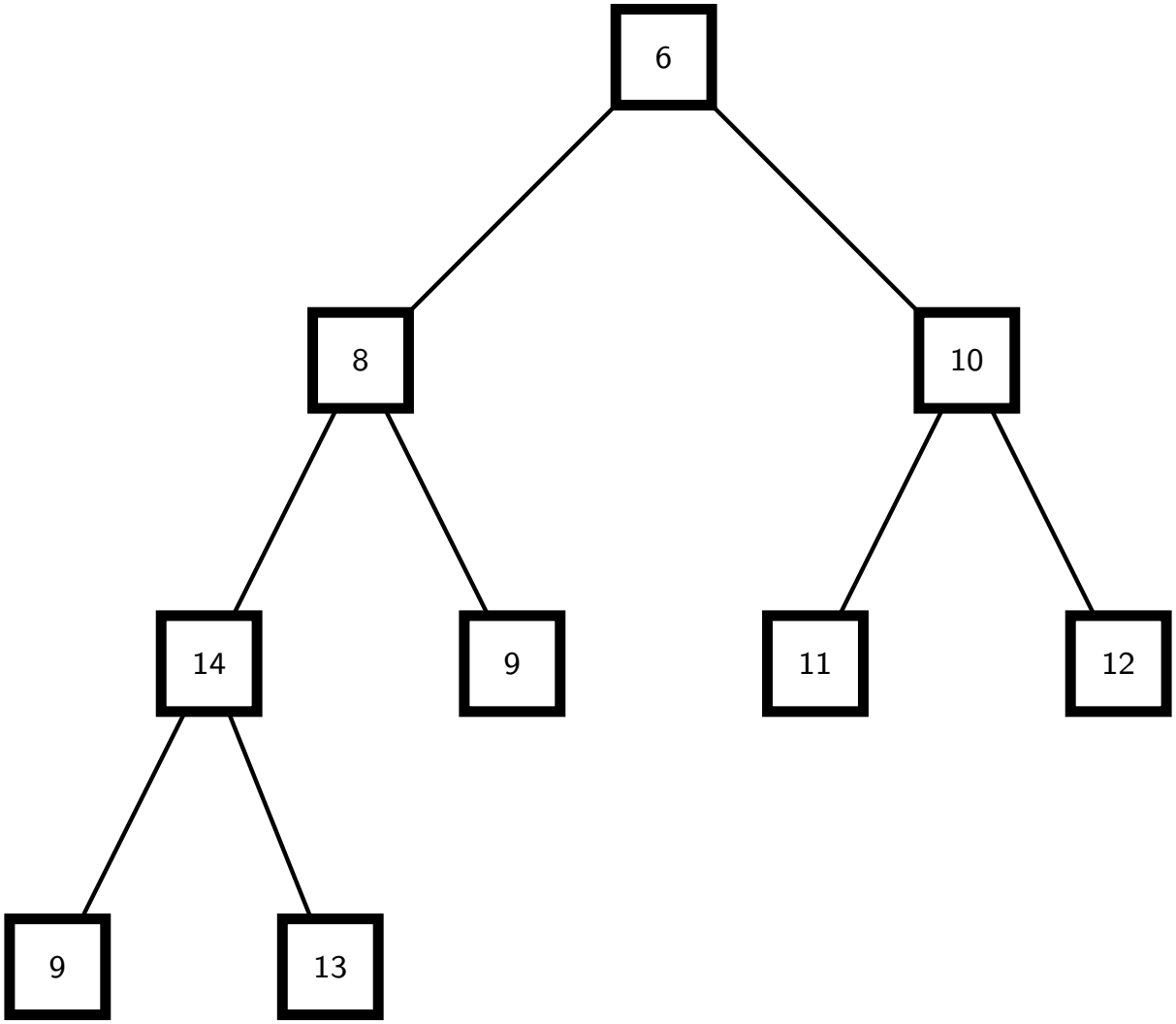


Suppression dans un tas : exemple

T =

1	2	3	4	5	6	7	8	9	10
6	14	10	8	9	11	12	9	13	14

Suppression dans un tas : exemple

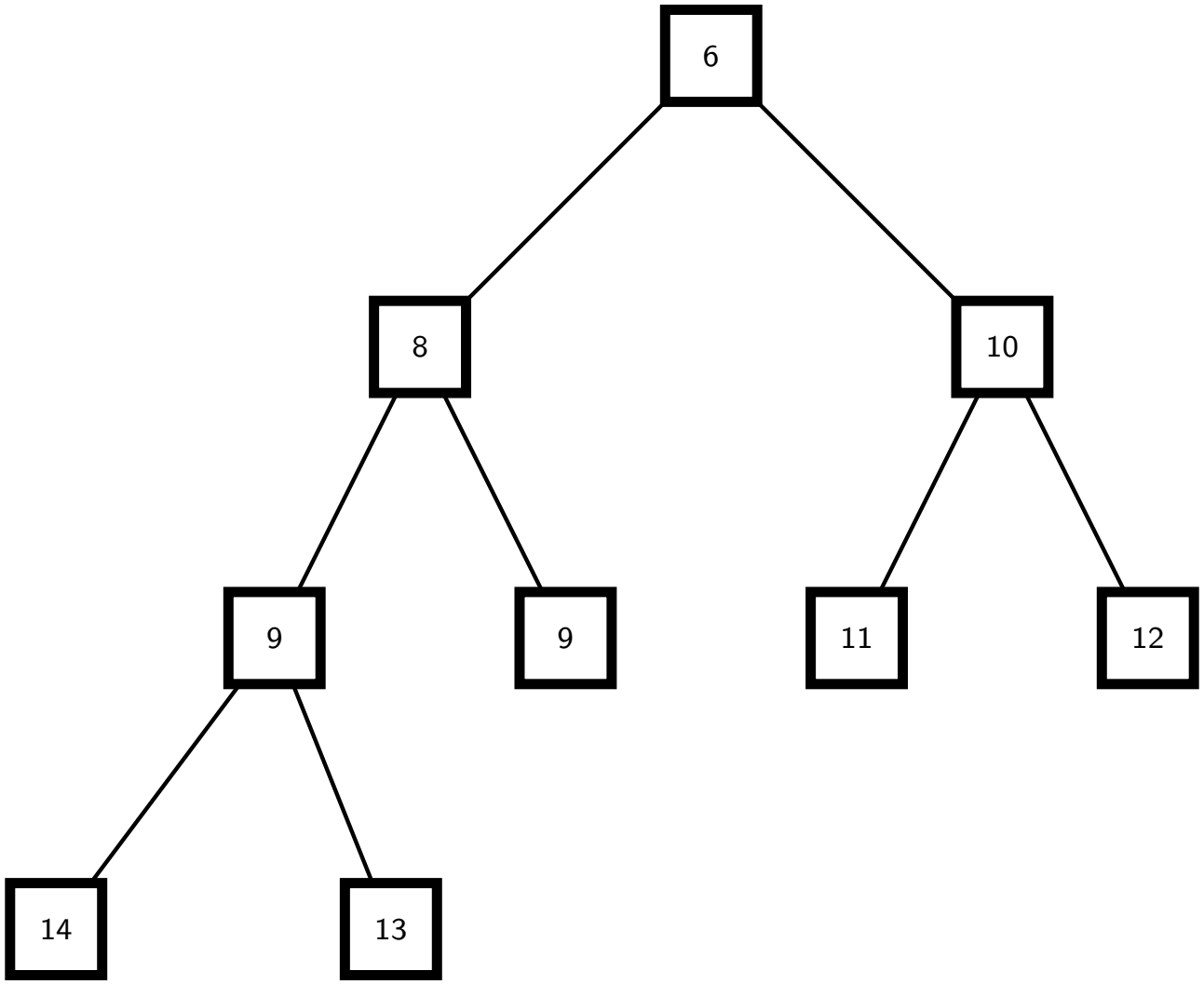


Suppression dans un tas : exemple

T =

1	2	3	4	5	6	7	8	9	10
6	8	10	14	9	11	12	9	13	14

Suppression dans un tas : exemple



Suppression dans un tas : exemple

T =

1	2	3	4	5	6	7	8	9	10
6	8	10	9	9	11	12	14	13	14

Suppression dans un tas : Réorganisation

La réorganisation peut se faire à partir de n'importe quel indice j de T comme suit.

```
1  Algorithme : Reorganiser(T, j, n)
2  i := j;
3  tant que i ≤ ⌊n/2⌋ faire
4      si T[2i] < T[2i + 1] alors
5          m := 2i;
6      fin
7      sinon
8          m := 2i + 1;
9      fin
10     si T[i] > T[m] alors
11         echanger(T[i], T[m]) ;
12         i := m;
13     fin
14     sinon
15         i := n;
16     fin
17 fin
```

Suppression dans un tas : Réorganisation

La réorganisation peut se faire à partir de n'importe quel indice j de T comme suit.

```
1  Algorithme : Reorganiser(T, j, n)
2  i := j;
3  tant que i ≤ ⌊n/2⌋ faire
4      si T[2i] < T[2i + 1] alors
5          m := 2i;
6      fin
7      sinon
8          m := 2i + 1;
9      fin
10     si T[i] > T[m] alors
11         echanger(T[i], T[m]) ;
12         i := m;
13     fin
14     sinon
15         i := n;
16     fin
17 fin
```

Nombre d'échanges = au pire la hauteur du tas.

Algorithme de suppression

Il est basé sur la réorganisation.

Algorithme : $Suppression(T, n)$

$min := T[1]$;
$T[1] := T[n]$;
$n := n - 1$;
$Reorganiser(T, 1, n)$;
$renvoyer\ min$;

Tri par tas

Si T est déjà un tas, le tri s'effectue comme suit.

Algorithme : $TriTas(T, n)$
$k = n$;
tant que $k > 1$ **faire**
  $min = Suppression(T, k)$;
  $k = k - 1$;
  $T[k + 1] = min$;
fin

Tri par tas

Si T n'est pas un tas, il faut au préalable le réorganiser pour qu'il le devienne. On a :

```
1  Algorithme : TriTas(T, n)
2  pour k = ⌊n/2⌋ ... 1 faire
3  |   Reorganiser(T, k, n) ;
4  fin
5  k = n ;
6  tant que k > 1 faire
7  |   min = Suppression(T, k) ;
8  |   k = k − 1;
9  |   T[k + 1] = min ;
10 fin
```

Tri par tas

Si T n'est pas un tas, il faut au préalable le réorganiser pour qu'il le devienne. On a :

Algorithme : $TriTas(T, n)$
pour $k = \lfloor \frac{n}{2} \rfloor$ ... 1 **faire**
  $Reorganiser(T, k, n)$;
fin
$k = n$;
tant que $k > 1$ **faire**
  $min = Suppression(T, k)$;
  $k = k - 1$;
  $T[k + 1] = min$;
fin

Nombre d'échanges = au pire n * (hauteur du tas).

Hauteur d'un arbre binaire parfait

Théorème

La hauteur d'un arbre binaire parfait de n noeuds est égale à $\lfloor log_2(n) \rfloor$

Les tris et leur complexité (à venir)

Nom		Complexité Moyenne
Tri Bulle	(Bubble Sort)	$O(n^2)$
Tri par insertion	(Insertion Sort)	$O(n^2)$
Tri Shell	(Shell Sort)	$O(n^2)$
Tri cocktail	(Shaker Sort)	$O(n^2)$
Tri fusion	(Merge Sort)	$O(nlog(n))$
Tri rapide	(Quick Sort)	$O(nlog(n))$
Tri par tas	(heap Sort)	$O(nlog(n))$

Tri fusion

Principe : Le tri fusion utilise la stratégie dite **"diviser pour régner"**.

- Etape 1 : On **divise** le tableau en deux sous-tableaux.
- Etape 2 : On **trie (par récursivité)** chaque sous-tableau.
- Etape 3 : On **fusionne** les deux sous-tableaux, pour obtenir un tableau complet trié.

Tri fusion

```
1 Algorithme : TriFusion(T,a,b)
2 si a < b alors
3 |   m = ⌊(a+b)/2⌋ ;
4 |   TriFusion(T,a,m) ;
5 |   TriFusion(T,m+1,b) ;
6 |   Fusion(T,a,m,b) ;
7 fin
```

Fusion

```
1  Algorithme : Fusion(T,a,m,b)
2  n_1 = m − a + 1 ; n_2 = b − m;
3  créer les tableaux G[1...n_1 + 1] et D[1...n_2 + 1] ;
4  pour i allant de 1 à n_1 faire
5  |    G[i] = T[a + i − 1] ;
6  fin
7  pour j allant de 1 à n_2 faire
8  |    D[j] = T[m + j] ;
9  fin
10 G[n_1 + 1] = ∞, D[n_2 + 1] = ∞ ;
11 i = 1 ; j = 1 ;
12 pour k allant de a à b faire
13 |    si G[i] ≤ D[j] alors
14 |    |    T[k] = G[i] ; i = i + 1 ;
15 |    fin
16 |    sinon
17 |    |    T[k] = D[j] ; j = j + 1 ;
18 |    fin
19 fin
```

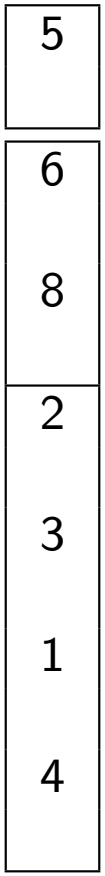
Exemple

5
6
8
2
3
1
4

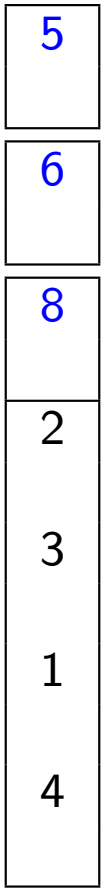
Exemple

5
6
8
2
3
1
4

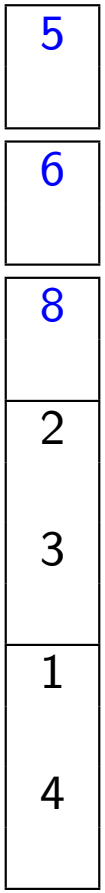
Exemple



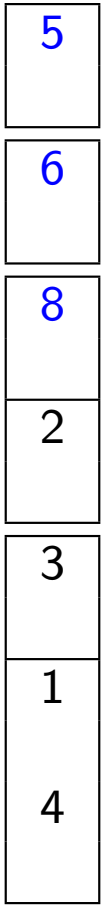
Exemple



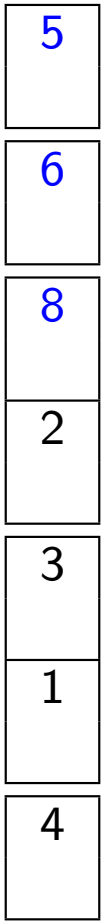
Exemple



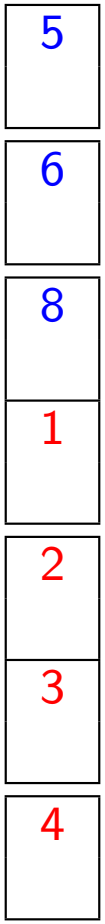
Exemple



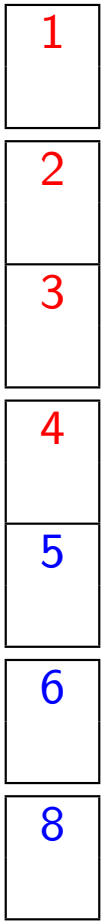
Exemple



Exemple



Exemple



Analyse du tri fusion en nombre de comparaisons

```
Algorithme : Fusion(T,a,m,b)
n_1 = m − a + 1 ; n_2 = b − m;
créer les tableaux G[1...n_1 + 1] et D[1...n_2 + 1] ;
pour i allant de 1 à n_1 faire
  |   G[i] = T[a + i − 1] ;
fin
pour j allant de 1 à n_2 faire
  |   D[j] = T[m + j] ;
fin
G[n_1 + 1] = ∞, G[n_2 + 1] = ∞ ;
i = 1 ; j = 1 ;
pour k allant de a à b faire
  |   si G[i] ≤ T[j] alors
  |     |   T[k] = G[i] ; i = i + 1 ;
  |   fin
  |   sinon
  |     |   T[k] = D[j] ; j = j + 1 ;
  |   fin
fin
```

Analyse du tri fusion en nombre de comparaisons

Soit $C(n)$ le nombre total, au pire des cas, des comparaisons indiquées en vert. On a :

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + n$$

Théorème

Si n est une puissance de 2 alors $C(n) = 2C(n/2) + n = n\ log_2(n)$

Démonstration.

□ □

Tri rapide

Principe : Soit T un tableau à trier. On se donne un élément quelconque du tableau (en général le premier à gauche) nommé pivot. On effectue une partition du tableau consistant à réorganiser les éléments de T de la façon suivante :

$$T[1...m-1], T[m], T[m+1...n]$$

Tri rapide

$$T[1...m-1], T[m], T[m+1...n]$$

- $T[m]$ contient le pivot choisi.
- $T[1...m-1]$ contient les éléments de T inférieurs à $T[m]$.
- $T[m+1...n]$ ceux supérieurs.
- Sur chaque sous-tableau, on réitére récursivement ce qui a été fait sur T.

Tri rapide

```
1 Algorithme : TriRapide(T,a,b)
2 si a < b alors
3 |   Partitionnement(T,a,b,m) ;
4 |   TriRapide(T,a,m-1) ;
5 |   TriRapide(T,m+1,b) ;
6 fin
```

Principe du partitionnement

- On considère comme pivot le premier élément du tableau en cours de traitement, $T[a]$.
- On utilise deux compteurs l et k initialisés aux deux extrémités (gauche pour l, et droite pour k) du tableau.
- Tant que $T[l] \leq T[a]$, l est incrémenté.
- Inversement tant que $T[k] > T[a]$, k est décrémenté.
- On échange alors $T[l]$ et $T[k]$
- On continue de façon analogue jusqu'à ce que les indices l et k se croisent.

Algorithme de partitionnement

```
1  Algorithme : Partionnement(T,a,b,m)
2  l = a + 1 ;
3  k = b ;
4  tant que l ≤ k faire
5  |   tant que T[l] ≤ T[a] faire
6  |   |   l = l + 1 ;
7  |   fin
8  |   tant que T[k] > T[a] faire
9  |   |   k = k − 1 ;
10 |   fin
11 |   si (l < k) alors
12 |   |   echanger(T[l], T[k]) ;
13 |   |   l = l + 1 ;
14 |   |   k = k − 1 ;
15 |   fin
16 fin
17 m = k ;
18 echanger(T[a], T[k]) ;
```

Exemple

pivot 1	5
l	6
	8
	2
	3
	1
k	4

Exemple

pivot 1	5
l	4
	8
	2
	3
	1
k	6

Exemple

pivot 1	5
	4
l	8
	2
	3
k	1
	6

Exemple

pivot 1	5
	4
l	1
	2
	3
k	8
	6

Exemple

pivot 1	5
	4
	1
l	2
k	3
	8
	6

Exemple

pivot 1	5
	4
	1
	2
k,l	3
	8
	6

Exemple

pivot 1	5
	4
	1
	2
k	3
l	8
	6

Exemple

pivot 1	3
	4
	1
	2
k	5
l	8
	6

Exemple

	3
	4
	1
	2
pivot 1	5
	8
	6

Exemple

pivot 2	3
l	4
	1
k	2
pivot 1	5
	8
	6

Exemple

pivot 2	3
l	2
	1
k	4
pivot 1	5
	8
	6

Exemple

pivot 2	3
	2
l,k	1
	4
pivot 1	5
	8
	6

Exemple

pivot 2	3
	2
k	1
l	4
pivot 1	5
	8
	6

Exemple

pivot 2	1
	2
k	3
l	4
pivot 1	5
	8
	6

Exemple

	1
	2
pivot 2	3
	4
pivot 1	5
	8
	6

Exemple

	1
	2
pivot 2	3
	4
pivot 1	5
pivot 3	8
	6

Exemple

	1
	2
pivot 2	3
	4
pivot 1	5
pivot 3	8
l,k	6

Exemple

	1
	2
pivot 2	3
	4
pivot 1	5
pivot 3	8
k	6

Exemple

	1
	2
pivot 2	3
	4
pivot 1	5
pivot 3	6
k	8

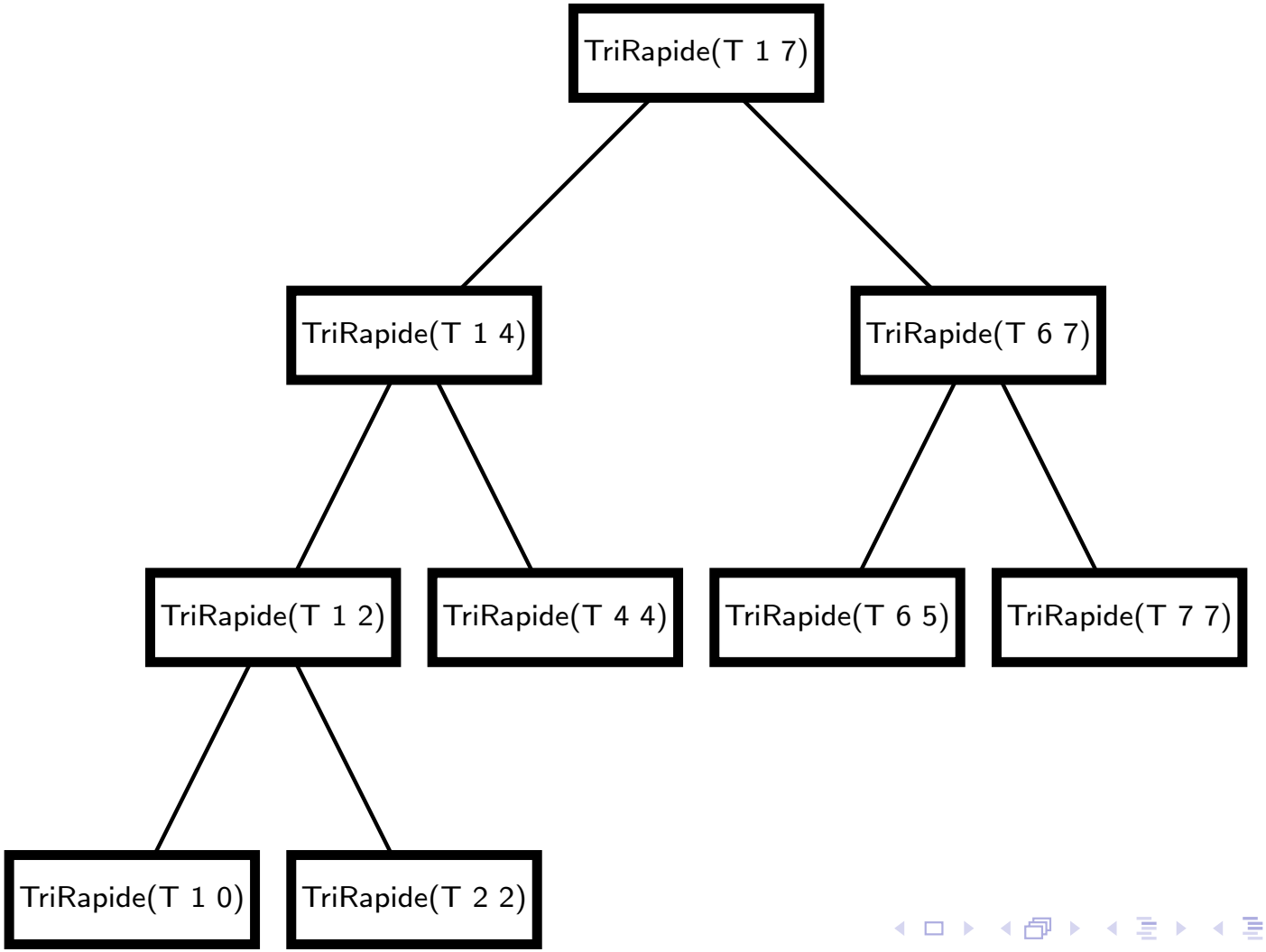
Dénombrement du nombre de comparaisons

Il y a $b - a + 2$ comparaisons.

```
Algorithme : Partionnement(T,a,b,m)
l = a + 1 ;
k = b ;
tant que l ≤ k faire
    tant que T[l] ≤ T[a] faire
        l = l + 1 ;
    fin
    tant que T[k] > T[a] faire
        k = k − 1 ;
    fin
    si (l < k) alors
        echanger(T[l], T[k]) ;
        l = l + 1 ;
        k = k − 1 ;
    fin
fin
m = k ;
echanger(T[a], T[k]) ;
```

Graphe des appels récursifs

Le graphe des appels récursifs du tri rapide est un **arbre binaire**.



Définitions

Définition

On appelle **complexité en temps** (resp. **en place mémoire**) une "mesure" du temps d'éxecution (resp. de la place mémoire) effectué (resp. prise) par un algorithme.

Définition

Une opération, "op", est dite **fondamentale**, pour un algorithme A, si la complexité en temps de A varie "proportionnellement" au nombre d'opérations "op" effectuées.

Tri bulles

```
Algorithme : TriBulles(T,n)
pour i = 1, ..., n − 1 faire
    pour j = n, ..., i + 1 faire
        si T[j] < T[j − 1] alors
            echanger T[j] et T[j − 1] ;
        fin
    fin
fin
```

Tri bulles : complexité en comparaisons

```
1 Algorithme : TriBulles(T,n)
2 pour i = 1, ..., n − 1 faire
3     pour j = n, ..., i + 1 faire
4         si T[j] < T[j − 1] alors
5             echanger T[j] et T[j − 1] ;
6         fin
7     fin
8 fin
```

$$\text{“}Comparaison\text{”} = T[j] < T[j-1] \text{ effectuée } \frac{n(n-1)}{2} \text{ fois}$$

Tri bulles : complexité en échanges au mieux, au pire

$$\text{“}Echange\text{”} = \text{echanger } T[j] \text{ et } T[j-1] \text{ effectuée ? fois}$$

- 0 fois au mieux,
- $\frac{n(n-1)}{2}$ au pire

Tri bulles : complexité en échanges en moyenne

Proposition

Le nombre total d'échanges est le nombre de couples d'indices (i, j) où $i < j$ dont les valeurs dans T ne sont pas dans le bon ordre (i.e $T[i] > T[j]$).

- Meilleur des cas : aucun couple dans le mauvais ordre. 0 échange.
- Pire des cas : tous les couples dans le mauvais ordre. $\frac{n(n-1)}{2}$ échanges.
- En moyenne : $\frac{n(n-1)}{4}$ échanges.

Notions de complexité au pire, au meilleur et en moyenne

Définition

Soit $C_A(d)$ la complexité en temps de l'algorithme A sur la donnée d et D_n l'ensemble des données de taille n. On appelle **complexité** :

- **dans le meilleur des cas** : $min_A(n) = min\{c_A(d)|d \in D_n\}$
- **dans le pire des cas** : $max_A(n) = max\{c_A(d)|d \in D_n\}$
- **en moyenne** : $moy_A(n) = \sum_{d \in D_n} prob(d)c_A(d)$

où $prob(d)$ est la probabilité que l'on ait l'instance d.

Tri rapide : complexité des comparaisons pire des cas

On s'intéresse aux comparaisons faites dans la procédure "partitionnement"

Proposition

T est déjà trié il y aura

$$\frac{(n+1)(n+2)}{2} - 3$$

comparaisons

Tri rapide : complexité des comparaisons meilleur des cas

- On s'intéresse toujours aux comparaisons faites dans la procédure "partitionnement"
- On suppose qu'à chaque itération le partitionnement divise le tableau en deux sous-listes de même taille

Proposition

Le nombre de comparaisons est dans ce cas

$$(n+1)(log_2(n+1)-1)$$

Résumé Analyse du tri bulles et insertion

Nombre de comparaisons

Tri	au mieux	au pire	en moyenne
Bulles	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{4}$
Rapide	$(n+1)(log_2(n+1)-1)$	$\frac{(n+1)(n+2)}{2}-3$	$\Theta(nlog_2(n))$

Notation asymptotique Θ

Définition

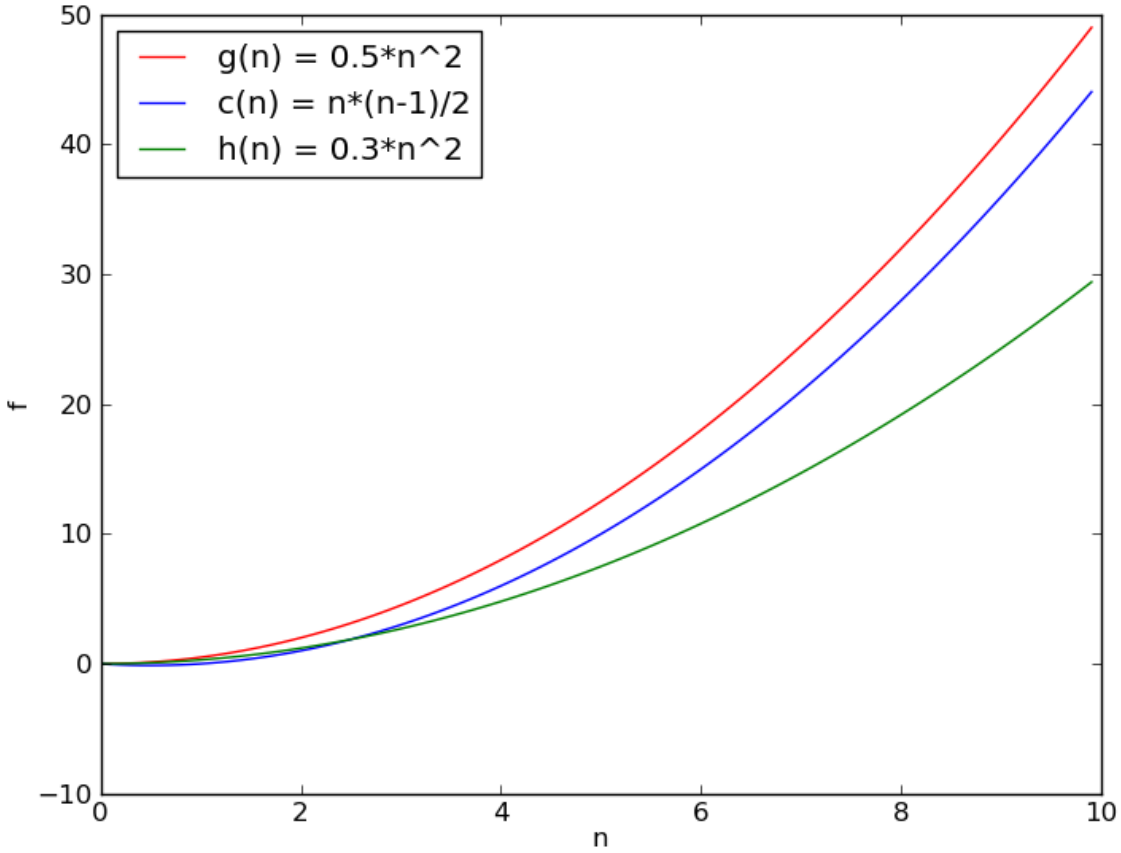
Soit $C(n)$ une fonction exprimant le temps d'exécution d'un algorithme pour des données de taille n. $C(n)$ et $g(n)$ ont même **ordre de grandeur asymptotique** ssi, à partir d'un certain rang N, on a l'encadrement :

$$cg(n) \leq C(n) \leq dg(n)$$

où c et d sont des constantes positives.

Notation asymptotique Θ

Exemple : $\frac{n(n-1)}{2} \in \Theta(n^2)$.



Notation (mathématique) asymptotique Θ

L'ensemble des fonctions qui ont même ordre de grandeur que $g(n)$ se note.

Définition

Soit $g : \mathbb{N} \to \mathbb{N}$. On note

$$\Theta(g(n)) = \{f : \mathbb{N} \to \mathbb{N} \mid \begin{array}{l} \exists\, c, d > 0\ , \\ \exists\, N \in \mathbb{N}\ ,\ cg(n) \leq f(n) \leq dg(n), \\ \forall\, n \geq N\} \end{array}$$

On note $C(n) \in \Theta(g(n))$ ou (par abus de langage) $C(n) = \Theta(g(n))$

Notation asymptotique O

Soit $g : \mathbb{N} \to \mathbb{N}$.

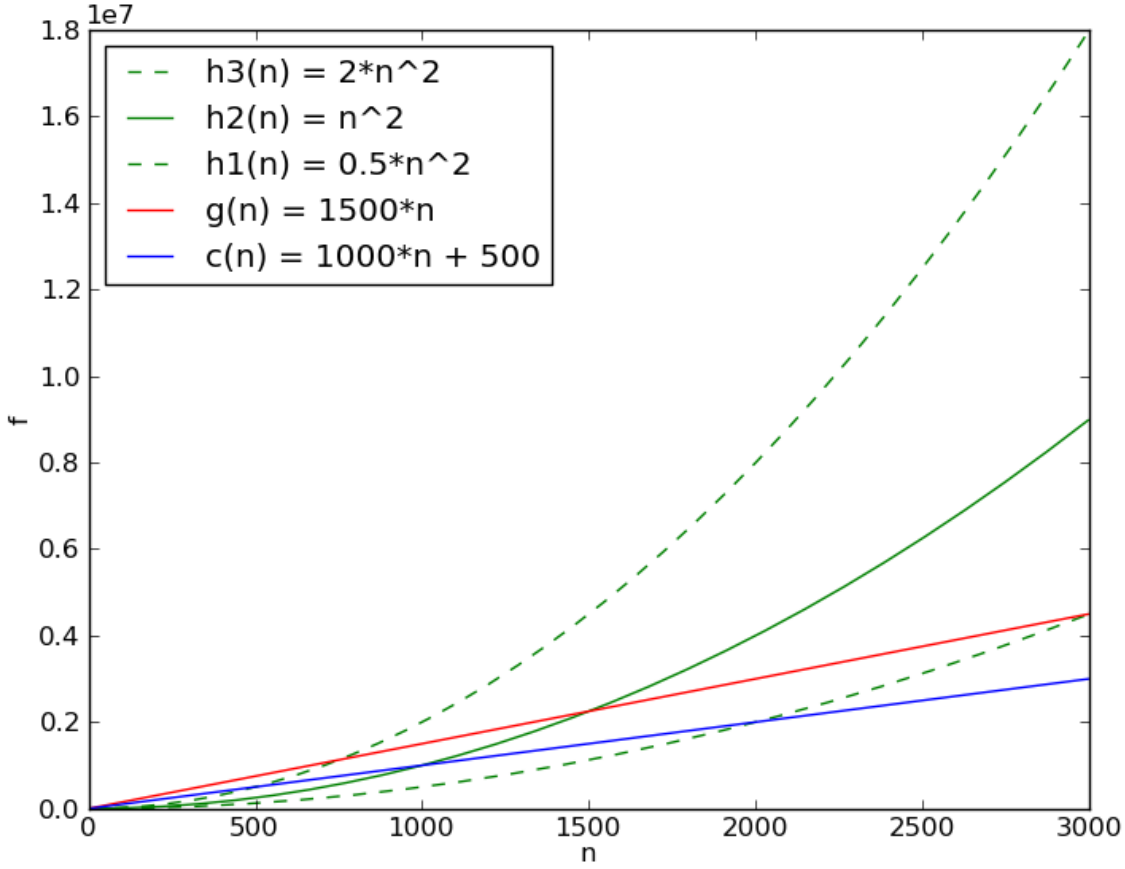
Définition

$$O(g(n)) = \{f : \mathbb{N} \to \mathbb{N} \mid \exists\, c > 0\,,\ \exists\, N \in \mathbb{N}\,,\ f(n) \leq cg(n),\ \forall\, n \geq N\}$$

Si $C(n) \in O(g(n))$, on dit que C est dominée asymptotiquement par g. On note aussi $C = O(g)$.

Exemple : $\frac{n(n-1)}{2} \in O(n^2)$.

Notation asymptotique O



Complexité de tris

Nombre de comparaisons.

Tri	au mieux	au pire	en moyenne
bulles	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Rapide	$\Theta(nlog_2(n))$	$\Theta(n^2)$	$\Theta(nlog_2(n))$
Tas	$O(nlog_2(n))$	$\Theta(nlog_2(n))$	$\Theta(nlog_2(n))$

Notation asymptotique Ω

Définition

Soit $g : \mathbb{N} \to \mathbb{N}$. On note

$$\Omega(g(n)) = \{f : \mathbb{N} \to \mathbb{N} \mid \exists\, c > 0\,,\ \exists\, N \in \mathbb{N}\,,\ cg(n) \leq f(n),\ \forall\, n \geq N\}$$

Si $C(n) \in \Omega(g(n))$, on dit que C domine asymptotiquement g. On note aussi $C = \Omega(g)$.

Exemple : $n^3 \in \Omega(n^2)$.

Echelles de comparaison

Certaines fonctions servent d'étalons de comparaison :

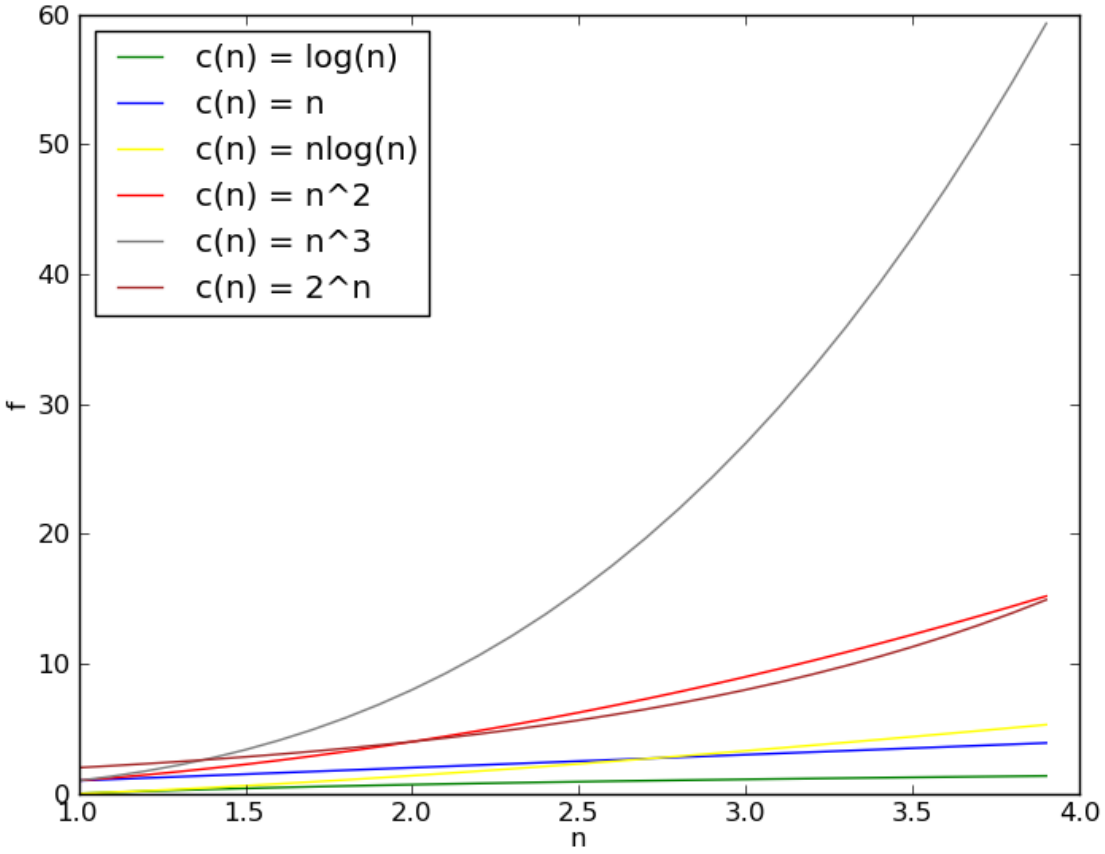
Théorème

$O(1) \subset O(log(n)) \subset O(n) \subset O(nlog(n)) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$, $\forall n \geq 2$

Echelles de comparaison

	complexité
$C = O(1)$	constante
$C = O(log(n))$	logarithmique
$C = O(n)$	linéaire
$C = O(n^2)$	quadratique
$C = O(n^3)$	cubique
$C = O(n^p)$	polynomiale
$C = O(2^n)$	exponentielle
$C = O(n!)$	exponentielle

Rapidité de croissance



Complexités polynomiales : quelques chiffres

Si $n = 10$. Si la machine réalise 1 opération par seconde

seconds	temps
10^2	1.7 min
10^4	2.8 heures
10^5	1.6 semaines
10^7	3.8 mois
10^8	3.1 ans
10^9	3.1 décennies
10^{10}	3.1 siècles
10^{11}	...

Complexités polynomiales : quelques chiffres

Si $n = 10^6$.

op par secondes	n	$nlog(n)$	n^2
10^6	secondes	secondes	semaines
10^9	instantané	instantané	heures
10^{12}	instantané	instantané	secondes

Backtracking (retour sur trace)

Définition

Le backtracking (retour sur trace en français) est une technique d'énumération de l'ensemble des décisions possibles d'un problème à résoudre.

Exemple : Sudoku

On dispose d'une grille 9 * 9 subdivisée en 9 blocs 3*3.

- Régle 1 : chaque cellule de cette grille doit comporter un chiffre entre 1 et 9.
- Régle 2 : un chiffre ne peut être plus d'une fois sur la même ligne, la même colonne ou le même bloc.

Exemple : Sudoku

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure – Tim Stellmach, Public domain, via Wikimedia Commons

Exemple Sudoku

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

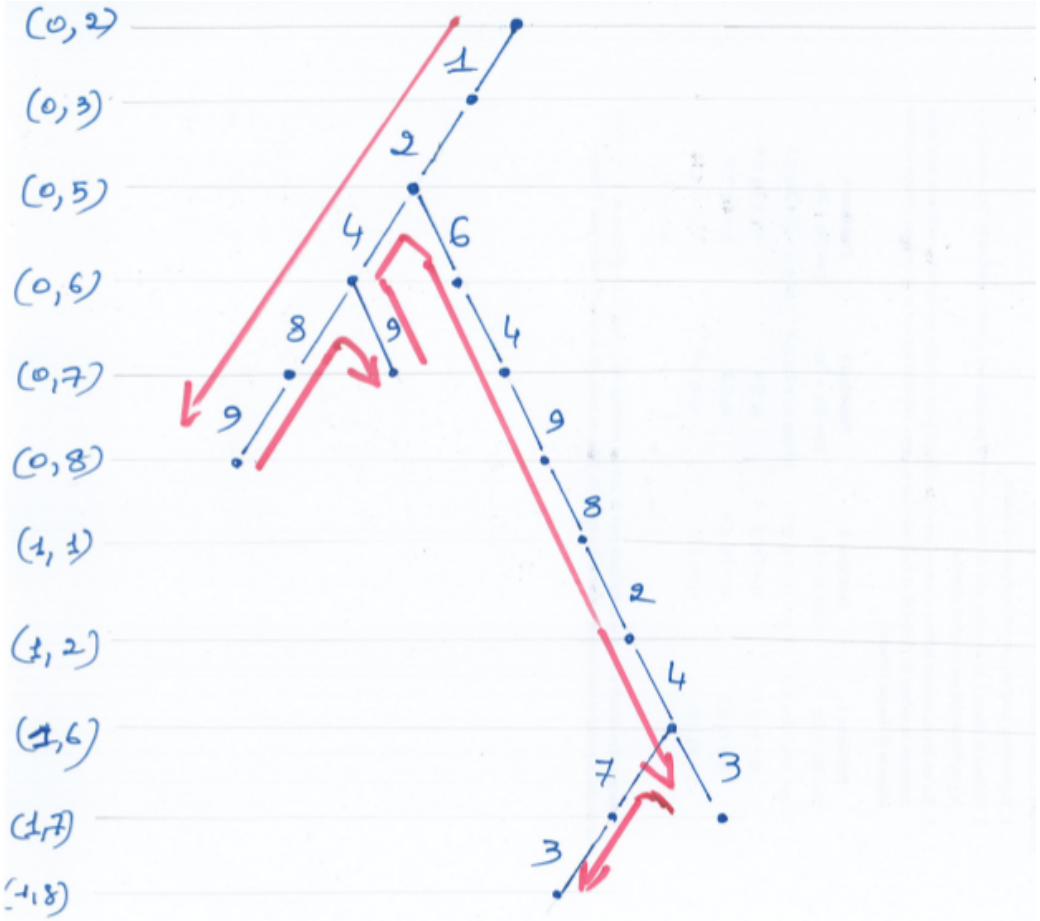
Résolution par backtracking

- Soit $r = 51$ (nombre de cellules à trouver dans l'exemple).
- On note $C = (C_1, C_2, ..., C_r)$ la liste des cellules vides de la grille : $(1,3); (1,4); (1,6); ...; (1,9); (2,2)...; (9,7)$.
- $C_i = k$ signifiera qu'on attribue à la cellule i = 1,2,...,r, le chiffre $k = 1, 2..., 9$.
- $C_i.l$ (resp. $C_i.c$, $C_i.b$) désignera le numéro de ligne (resp. colonne, bloc) de la cellule C_i

Backtracking Sudoku

```
1  Algorithme : BacktrackingSudoku(C,k)
2  si k == r+1 alors
3  |    retourner(vrai);
4  fin
5  sinon
6  |    pour j allant de 1 à 9 faire
7  |    |    si j n'est ni sur la ligne C_k.l, ni sur la colonne C_k.c, ni sur le bloc C_k.b alors
8  |    |    |    grille[C_k.l][C_k.c] = j;
9  |    |    |    si BacktrackingSudoku(C,k+1) == vrai alors
10 |    |    |    |    retourner(vrai);
11 |    |    |    fin
12 |    |    |    grille[C_k.l][C_k.c] = 0;
13 |    |    fin
14 |    fin
15 fin
16 retourner(faux);
```

Arbre d'énumération exhaustive



Algorithmes Min-Max

Définition

Les algorithmes min-max sont des variantes de backtracking utilisées dans les jeux à (au moins) deux joueurs. Ils sont basés sur des appels **récursifs croisés**.

Récursivité simple

```
1 Algorithme : f(A)
2 si A vérifie une condition d'arrêt donnée alors
3 |   Fin de l'algorithme;
4 fin
5 sinon
6 |   ...;
7 |   f(B);
8 |   ...;
9 fin
```

Récursivité croisée

```
1 Algorithme : f(A)
2 si A vérifie une condition d'arrêt donnée alors
3 |   Fin de l'algorithme;
4 fin
5 sinon
6 |   g(B);
7 |   ...;
8 fin
```

```
1 Algorithme : g(A)
2 si A vérifie une condition d'arrêt donnée alors
3 |   Fin de l'algorithme;
4 fin
5 sinon
6 |   f(B);
7 |   ...;
8 fin
```

Morpion (Tic-Tac-Toe)

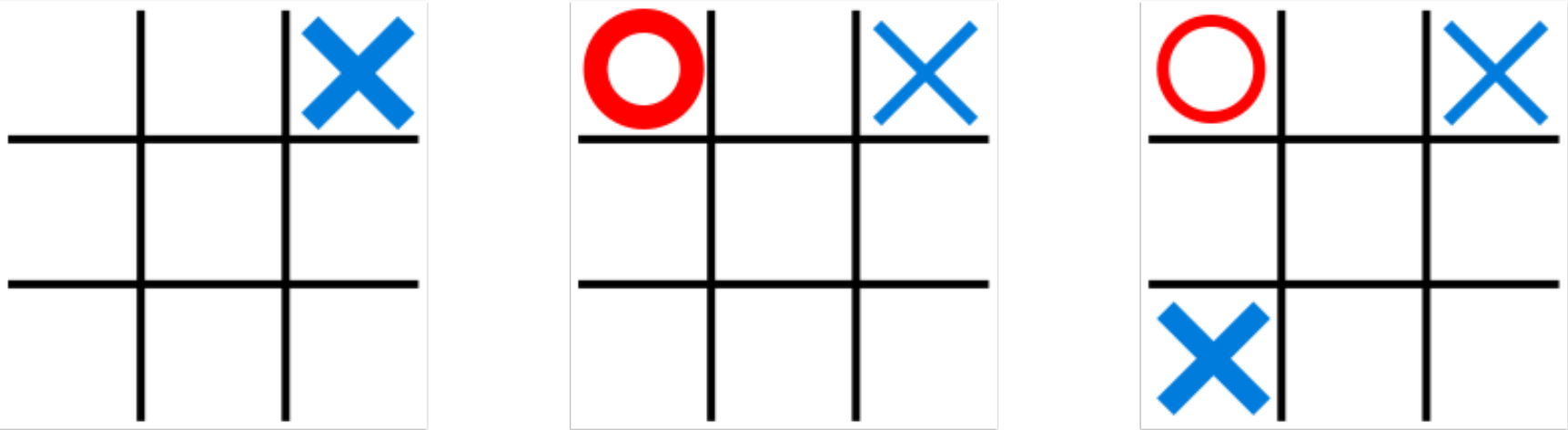


Figure – Source : wikipedia

Morpion (Tic-Tac-Toe)

On représente la partie par un tableau : int T[3][3]. On note :

- ORDI : valeur entière quelconque représentant l'ordinateur (+1).
- HUMAIN : valeur entière quelconque représentant l'humain (-1).
- Une partie est soit gagnée par l'ordinateur (+1), perdue par l'ordinateur (-1 : gagnée par l'humain), ou nul (0)

Etat terminal

Définition

On appelle état terminal du jeu, un état du tableau T pour lequel on peut dire si l'ordinateur a gagné ($+1$), perdu (-1) ou si c'est un match nul (0)

Définition

La valeur d'un état terminal est $+1$, 0 ou -1

Exemple Etat terminal

-1	-1	1
-1	1	
1		

Exemple Etat terminal

-1	1	-1
-1	1	1
1	-1	-1

Exemple Etat Non-terminal

-1	-1	
-1	1	
1		

Stratégie Min-Max

Définition

La valeur d'un état non-terminal est déterminée récursivement en supposant que chaque joueur a une stratégie optimale.

Par "optimale", on entend :

- L'ordinateur cherche à jouer des coups menant à sa victoire (+1). Il cherche donc la position de valeur maximale (max).
- L'humain cherche à jouer des coups menant à sa victoire (-1). Il cherche donc la position de valeur minimale (min).

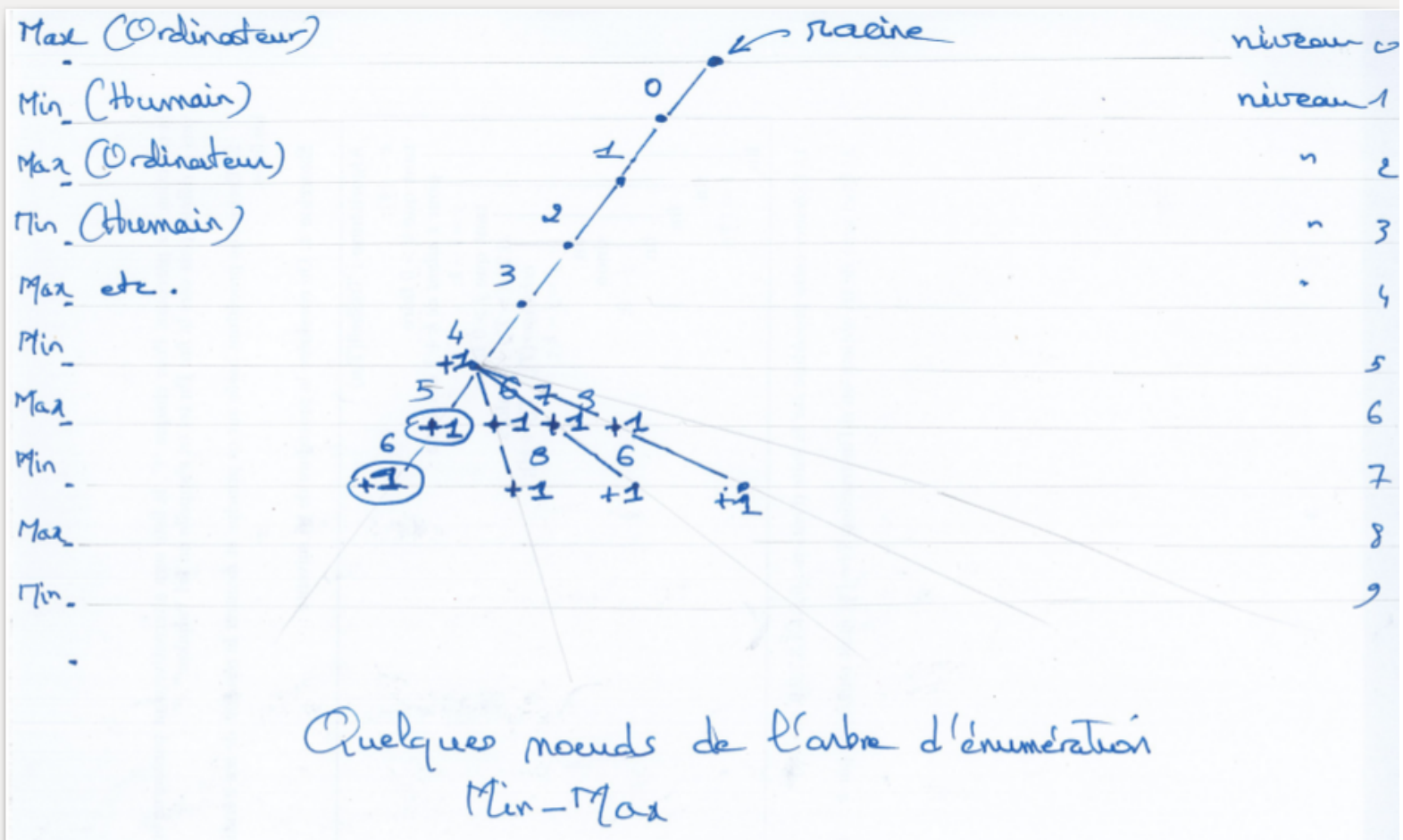
Stratégie "Max" de l'ordinateur

```
1  Algorithme : JeuOrdi(bestMove)
2  arg : valeur entière;
3  si plein() alors
4  |    retourner(NUL);
5  fin
6  si coupgagnant(ORDI,bestMove) alors
7  |    retourner(VORDI);
8  fin
9  val = VHUMAIN;
10 pour i = 0, 1..., 8 faire
11 |    si vide(i) alors
12 |    |    jouer(i,ORDI);
13 |    |    res = JeuHumain(arg);
14 |    |    retourarriere(i);
15 |    |    si res > val alors
16 |    |    |    val = res;
17 |    |    |    bestMove = i;
18 |    |    fin
19 |    fin
20 fin
21 retourner(val);
```

Stratégie "Min" de l'humain

```
1  Algorithme : JeuHumain(bestMove)
2  arg : valeur entière;
3  si plein() alors
4  |    retourner(NUL);
5  fin
6  si coupgagnant(HUMAIN,bestMove) alors
7  |    retourner(VHUMAIN);
8  fin
9  val = VORDI;
10 pour i = 0, 1..., 8 faire
11 |    si vide(i) alors
12 |    |    jouer(i,HUMAIN);
13 |    |    res = JeuOrdi(arg);
14 |    |    retourarriere(i);
15 |    |    si res < val alors
16 |    |    |    val = res;
17 |    |    |    bestMove = i;
18 |    |    fin
19 |    fin
20 fin
21 retourner(val);
```

Arbre d'énumération exhaustive



Nombre d'états

- Si l'ordinateur joue en premier il examinera 97612 états.
- Si l'humain joue au centre, la machine examinera 5185 états.
- Si l'humain joue une case en coin, l'ordinateur examinera 9761 états.